

Stack and Queue

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Stack and Queue	1
1.1	Stacks, Queues, and the Monotonic Stack Idea	1
1.2	Implement a Stack using an Array	3
1.3	Implement a Stack using a Linked List	4
1.4	Implement a Queue using a Linked List	6
1.5	Implement a Stack using a Queue	8
1.6	Implement a Queue using Stacks	10
1.7	Min Stack	13
1.8	Valid Parentheses	15
1.9	Infix to Postfix Conversion	17
1.10	Infix to Prefix Conversion	20
1.11	Postfix to Infix Conversion	23
1.12	Postfix to Prefix Conversion	25
1.13	Prefix to Infix Conversion	26
1.14	Prefix to Postfix Conversion	28
1.15	Next Greater Element I	30
1.16	Next Greater Element II (Circular Array)	32
1.17	Next Smaller Element	35
1.18	Number of NGEs to the Right	37
1.19	Trapping Rain Water	39
1.20	Sum of Subarray Minimums	42
1.21	Sum of Subarray Ranges	44
1.22	Asteroid Collision	47
1.23	Remove K Digits	49
1.24	Largest Rectangle in Histogram	52
1.25	Maximal Rectangle	54
1.26	Stock Span Problem	56
1.27	The Celebrity Problem	59
1.28	Sliding Window Maximum	61
1.29	LRU Cache	64
1.30	LFU Cache	67

Chapter 1

Stack and Queue

A stack remembers what just happened. A queue remembers what happened first. Almost every problem in this chapter is, underneath, a decision about which of these two memories the problem actually needs – and a surprising number of “queue” problems turn out to be built from stacks, and vice versa.

1.1 Stacks, Queues, and the Monotonic Stack Idea

A **stack** supports two operations, *push* and *pop*, with Last-In-First-Out (LIFO) order: the most recently pushed element is always the first one popped. A **queue** supports *enqueue* and *dequeue* with First-In-First-Out (FIFO) order: the earliest enqueued element still present is always the first one dequeued. Both can be implemented on top of an array, a linked list, or – as the first section of this chapter shows – on top of *each other*.

How to Recognise Which One You Need

- **“Matching/nesting/balanced”** (parentheses, expression evaluation, undo history) – a **stack**, because the most recent unmatched thing is always the first thing that needs to be closed or undone.
- **“Next/previous greater/smaller element”** – a **monotonic stack**: a stack kept strictly increasing or decreasing from bottom to top by popping elements that violate the order before pushing a new one. This is the single most important pattern in this chapter, covered in depth starting at Section 1.15.
- **“Maximum/minimum over every sliding window”** – a **monotonic deque** (Section 1.28): a double-ended queue that behaves like a monotonic stack at one end, while also supporting eviction from the other end as the window slides.
- **“Most/least recently used,” with fast lookup** – a combination of a hash map (for $O(1)$ lookup) and a doubly linked list or queue-like ordering (for fast reordering by recency), as in LRU/LFU Cache (Sections 1.29–1.30).

The Monotonic Stack, Explained Once

Why a Monotonic Stack Finds “Next Greater” in $O(n)$

To find, for every element, the next element to its right that is strictly greater, scan from **right to left**, maintaining a stack that is kept strictly *decreasing* from bottom to top. Before processing element i , pop every stack element that is $\leq arr[i]$ – those elements can never again be the “next greater” answer for anything to their left, because $arr[i]$ is closer and at least as large, so it would always be found first; equivalently they have just been proven useless once $arr[i]$ shows up. After popping, whatever remains on top of the stack (if anything) is the next greater element for $arr[i]$ – the nearest still-undefeated candidate to i ’s right. Then push $arr[i]$ itself, since it might be the answer for elements further to the

left.

The reason this is $O(n)$ total, not $O(n^2)$: every element is pushed exactly once and popped at most once, across the *entire* scan – so even though some steps pop many elements, the total number of pops across all steps combined can never exceed n .

This single idea – maintain a monotonic stack, popping whatever the current element invalidates before pushing – powers roughly a third of this chapter’s problems, sometimes directly (next/previous greater or smaller element) and sometimes as a hidden sub-step inside a larger problem (histogram area, trapping rain water, sum of subarray minimums).

Each section in this chapter follows the same structure as the rest of this book: problem statement and keywords; the reasoning that identifies the right stack/queue technique and why it is correct; the algorithm in words and pseudocode; a hand-traced dry run; complete compilable C++ code; and a time/space complexity analysis.

1.2 Implement a Stack using an Array

Problem Statement. Implement a stack supporting *push(x)*, *pop()*, *top()*, and *isEmpty()*, backed by a fixed-size array, each operation running in $O(1)$.

Pattern Recognition

No pattern recognition is needed here – this is foundational machinery. The only idea worth internalizing is that a stack needs just **one** extra piece of state beyond the array itself: an index, conventionally called *top*, pointing at the most recently pushed element.

Understanding the problem carefully

Since a stack only ever adds or removes from one end (the “top”), an array can simulate it perfectly by treating one end (conventionally the highest used index) as that top, and tracking that index explicitly. No shifting of other elements is ever required, which is exactly what keeps every operation $O(1)$.

The Single Pointer Invariant

Maintain *top* as the index of the most recently pushed element (-1 when empty). *push(x)* writes x at index $top + 1$ and increments *top*. *pop()* reads $arr[top]$ and decrements *top* (the old value is left in place but is considered logically removed, since future pushes will overwrite it).

Algorithm

1. $top \leftarrow -1$ initially.
2. *push(x)*: increment *top*; set $arr[top] \leftarrow x$.
3. *pop()*: read $arr[top]$; decrement *top*; return the read value.
4. *top()*: return $arr[top]$.
5. *isEmpty()*: return $(top = -1)$.

Dry run

Dry Run. Push 10, 20, 30; pop once; push 40.

- Push 10: $top = 0$, $arr = [10]$.
- Push 20: $top = 1$, $arr = [10, 20]$.
- Push 30: $top = 2$, $arr = [10, 20, 30]$.
- Pop: returns 30; $top = 1$.
- Push 40: $top = 2$, $arr = [10, 20, 40]$ (index 2 is overwritten).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Stack {
5     vector<int> arr;
6     int topIdx;
7
8 public:
9     Stack(int capacity) : arr(capacity), topIdx(-1) {}
10
11     void push(int x) {
12         arr[++topIdx] = x;
13     }
14
15     int pop() {
16         return arr[topIdx--];
17     }
18
19     int top() {
20         return arr[topIdx];
21     }
22
23     bool isEmpty() {
24         return topIdx == -1;
25     }
26 };
27
28 int main() {
29     Stack s(10);
30     s.push(10); s.push(20); s.push(30);
31     cout << "Popped: " << s.pop() << endl;
32     s.push(40);
33     cout << "Top: " << s.top() << endl;
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. $O(1)$ for every operation – only a single index update and a single array access are involved. **Space Complexity.** $O(\text{capacity})$ for the underlying array.

Key Takeaway

A stack needs exactly one pointer into a contiguous array. Compare this to Section 1.4, where a queue's two-ended access pattern makes a plain array awkward (without wraparound) and motivates a linked-list-based implementation instead.

1.3 Implement a Stack using a Linked List

Problem Statement. Implement a stack supporting *push(x)*, *pop()*, *top()*, and *isEmpty()*, backed by a singly linked list, each operation running in $O(1)$, with no fixed capacity limit.

Pattern Recognition

The keyword shift from Section 1.2 is “*no fixed capacity*” – an array-backed stack needs to know its maximum size up front (or pay for resizing), while a linked list grows and shrinks node by node with no such limit. The structural insight: a stack’s LIFO order matches a linked list’s **head** perfectly, since inserting and removing at the head of a singly linked list are both already $O(1)$.

Understanding the problem carefully

If we always insert new elements at the **head** of the list and always remove from the head, the most recently inserted element is always the first one removed – exactly LIFO order, with no need to traverse the list at all.

Head Insertion = Push, Head Removal = Pop

Maintain a single pointer *head* to the front of the list (null when empty). *push(x)* creates a new node with $next \leftarrow head$, then sets *head* to this new node. *pop()* reads *head*’s value, then advances $head \leftarrow head.next$.

Algorithm

1. $head \leftarrow \text{null}$ initially.
2. $push(x)$: $node \leftarrow \text{new node with value } x$, $next \leftarrow head$; $head \leftarrow node$.
3. $pop()$: $val \leftarrow head.val$; $head \leftarrow head.next$; return *val*.
4. $top()$: return $head.val$.
5. $isEmpty()$: return $(head = \text{null})$.

Dry run

Dry Run. Push 1, 2, 3; pop once.

- Push 1: list = 1 $\rightarrow$ null.
- Push 2: list = 2 $\rightarrow$ 1 $\rightarrow$ null.
- Push 3: list = 3 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ null.
- Pop: returns 3; list = 2 $\rightarrow$ 1 $\rightarrow$ null.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Stack {
5     struct Node {
6         int val;
7         Node* next;
8         Node(int v, Node* n) : val(v), next(n) {}
9     };
10    Node* head = nullptr;

```

```

11
12 public:
13     void push(int x) {
14         head = new Node(x, head);
15     }
16
17     int pop() {
18         int val = head->val;
19         Node* old = head;
20         head = head->next;
21         delete old;
22         return val;
23     }
24
25     int top() {
26         return head->val;
27     }
28
29     bool isEmpty() {
30         return head == nullptr;
31     }
32 };
33
34 int main() {
35     Stack s;
36     s.push(1); s.push(2); s.push(3);
37     cout << "Popped: " << s.pop() << endl;
38     cout << "Top: " << s.top() << endl;
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. $O(1)$ for every operation – all of them touch only the head node.

Space Complexity. $O(n)$ for n stored elements, with one node of constant-size overhead each, and no wasted unused capacity (unlike a fixed-size array implementation).

Key Takeaway

A stack's LIFO discipline matches a singly linked list's head exactly – no *tail* pointer is ever needed, since insertion and removal both happen at the same end. This is the cleanest possible linked-list data structure to implement, and it is exactly why the next section's queue (needing access at *both* ends) is structurally harder.

1.4 Implement a Queue using a Linked List

Problem Statement. Implement a queue supporting *enqueue(x)*, *dequeue()*, *front()*, and *isEmpty()*, backed by a singly linked list, each operation running in $O(1)$.

Pattern Recognition

A queue's FIFO discipline needs insertion at **one** end and removal from the **other** – unlike a stack, a single *head* pointer is not enough, because removing from the head and inserting at the head would give LIFO, not FIFO. The keyword to notice: this is the first structure in this chapter that genuinely requires **two** pointers into the list (*head* and *tail*) to stay $O(1)$.

Understanding the problem carefully

If we insert new elements at the **tail** and remove from the **head**, the earliest-inserted, not-yet-removed element is always at the head, ready to be dequeued – exactly FIFO order. Maintaining only a *head* pointer would make tail-insertion cost $O(n)$ (we would have to walk the whole list to find the current last node), so we keep an explicit *tail* pointer too, updated on every enqueue.

Two Pointers: Head for Removal, Tail for Insertion

enqueue(x) creates a new node and links it after the current *tail* (or sets it as both *head* and *tail* if the queue was empty), then updates *tail* to point to this new node. *dequeue()* reads *head*'s value, then advances $head \leftarrow head.next$ (and resets *tail* to null if the queue becomes empty).

Algorithm

1. *head, tail* $\leftarrow$ null initially.
2. *enqueue(x)*: *node* $\leftarrow$ new node with value *x*. If *tail* = null: *head* $\leftarrow$ *tail* $\leftarrow$ *node*. Else: *tail.next* $\leftarrow$ *node*; *tail* $\leftarrow$ *node*.
3. *dequeue()*: *val* $\leftarrow$ *head.val*; *head* $\leftarrow$ *head.next*; if *head* = null: *tail* $\leftarrow$ null; return *val*.
4. *front()*: return *head.val*.
5. *isEmpty()*: return (*head* = null).

Dry run

Dry Run. Enqueue 1, 2, 3; dequeue once.

- Enqueue 1: *head* = *tail* = 1.
- Enqueue 2: $1 \rightarrow 2$; *head* = 1, *tail* = 2.
- Enqueue 3: $1 \rightarrow 2 \rightarrow 3$; *head* = 1, *tail* = 3.
- Dequeue: returns 1; *head* = 2 now; list $2 \rightarrow 3$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Queue {
5     struct Node {
6         int val;
7         Node* next;
8         Node(int v) : val(v), next(nullptr) {}
9     };
10    Node* head = nullptr;
11    Node* tail = nullptr;
12
13 public:
14     void enqueue(int x) {
15         Node* node = new Node(x);
16         if (!tail) {

```

```

17         head = tail = node;
18     } else {
19         tail->next = node;
20         tail = node;
21     }
22 }
23
24 int dequeue() {
25     int val = head->val;
26     Node* old = head;
27     head = head->next;
28     if (!head) tail = nullptr;
29     delete old;
30     return val;
31 }
32
33 int front() {
34     return head->val;
35 }
36
37 bool isEmpty() {
38     return head == nullptr;
39 }
40 };
41
42 int main() {
43     Queue q;
44     q.enqueue(1); q.enqueue(2); q.enqueue(3);
45     cout << "Dequeued: " << q.dequeue() << endl;
46     cout << "Front: " << q.front() << endl;
47     return 0;
48 }

```

Complexity Analysis

Time Complexity. $O(1)$ for every operation, since *tail* removes the need to traverse the list on enqueue. **Space Complexity.** $O(n)$ for n stored elements.

Key Takeaway

The need for two pointers (*head* and *tail*) rather than one is the structural signature of FIFO order on a singly linked list. This same head/tail pairing reappears, in a different form, inside the doubly linked list used by LRU Cache (Section 1.29).

1.5 Implement a Stack using a Queue

Problem Statement. Implement a stack (LIFO: *push*, *pop*, *top*) using only the standard operations of a queue (*enqueue*, *dequeue*, *front*) as building blocks – no direct array or linked-list indexing allowed.

Pattern Recognition

“Implement structure X using only structure Y ’s operations” is a request to simulate one access order using a tool built for the opposite access order. The trick, here and in Section 1.6, is realizing that you can pay an **extra cost at one specific operation** (either

push or pop, your choice) to physically reorder the underlying structure's contents, restoring the access pattern you actually need.

Understanding the problem carefully

A queue always exposes its *oldest* element at the front. A stack needs to expose its *newest* element at the top. If, every time we push a new element, we immediately rotate the entire queue so that the new element ends up at the front (instead of the back, where a plain enqueue would place it), then the front of the queue always holds the most recently pushed element – exactly what a stack's *top* needs.

Make Push Expensive, Keep Pop/Top Cheap

push(x): enqueue x normally (it lands at the back). Then rotate the queue by dequeuing and re-enqueuing every *other* element (everything that was already in the queue before x was added) – this moves x , one rotation step at a time, all the way around to the front. *pop()* and *top()* then become trivial: simply dequeue (or peek) the queue's front, which is always the most recently pushed element by the invariant just established.

Why rotating after every push maintains the LIFO invariant

Each push performs exactly (*size before this push*) rotation steps, which is enough to move the newly enqueued element from the back to the front while preserving the relative order of everything else (each older element is dequeued and immediately re-enqueued once, so their relative order among themselves never changes). After this rotation, the queue's front-to-back order is exactly “most recently pushed first,” i.e. LIFO order, by induction: if this invariant held before the push, the rotation restores it after the push as well.

Algorithm

1. *push(x)*: let s be the queue's current size before this push. Enqueue x . Repeat s times: dequeue an element and immediately re-enqueue it.
2. *pop()*: dequeue and return the front.
3. *top()*: return the front, without removing it.

Dry run

Dry Run. Push 1, 2, 3 in order.

- Push 1: queue = [1] ($s = 0$, no rotation needed).
- Push 2: enqueue 2: queue = [1, 2]. Rotate $s = 1$ time: dequeue 1, enqueue 1: queue = [2, 1].
- Push 3: enqueue 3: queue = [2, 1, 3]. Rotate $s = 2$ times: dequeue 2, enqueue 2: queue = [1, 3, 2]; dequeue 1, enqueue 1: queue = [3, 2, 1].

The queue's front-to-back order is [3, 2, 1] – exactly LIFO order, most recently pushed (3) at the front.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class StackUsingQueue {
5     queue<int> q;
6
7 public:
8     void push(int x) {
9         int s = q.size();
10        q.push(x);
11        for (int i = 0; i < s; i++) {
12            q.push(q.front());
13            q.pop();
14        }
15    }
16
17    int pop() {
18        int val = q.front();
19        q.pop();
20        return val;
21    }
22
23    int top() {
24        return q.front();
25    }
26 };
27
28 int main() {
29     StackUsingQueue s;
30     s.push(1); s.push(2); s.push(3);
31     cout << "Top: " << s.top() << endl;
32     cout << "Popped: " << s.pop() << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. *push* costs $O(n)$ (the rotation touches every existing element). *pop* and *top* cost $O(1)$. **Space Complexity.** $O(n)$ for the underlying queue.

Key Takeaway

When forced to simulate one access order using a structure built for the opposite order, you must pay an extra reordering cost *somewhere* – the only design choice is *which* operation absorbs that cost. Here we chose to make *push* expensive and *pop/top* cheap; Section 1.6 makes the opposite choice for the reverse simulation, illustrating both options.

1.6 Implement a Queue using Stacks

Problem Statement. Implement a queue (FIFO: *enqueue*, *dequeue*, *front*) using only the standard operations of a stack (*push*, *pop*, *top*) as building blocks.

Pattern Recognition

The reverse simulation of Section 1.5. The key fact to recall: reversing a sequence **twice** restores its original order – so pouring elements through *two* stacks in succession (stack $\rightarrow$ stack $\rightarrow$ stack) flips LIFO order into FIFO order, because each individual pour through a stack reverses the order once.

Understanding the problem carefully

A single stack inherently reverses whatever order you feed into it: if you push 1, 2, 3 and then pop everything, you get 3, 2, 1 – reversed. If we use **two** stacks, call them *inStack* and *outStack*, and whenever *outStack* runs empty we pour all of *inStack* into it (popping from *inStack* and pushing each onto *outStack*), the act of pouring reverses the order a *second* time – and two reversals cancel out, restoring the original (FIFO) order on *outStack*'s pop sequence.

Two Stacks: One for Input, One for Output

enqueue(x): simply push x onto *inStack* – $O(1)$, no reordering needed yet. *dequeue()* (and *front()*): if *outStack* is empty, pour the entirety of *inStack* into it (pop each element off *inStack*, push it onto *outStack*); this reverses *inStack*'s order, placing the *oldest-enqueued*, *not-yet-dequeued* element on top of *outStack*. Then pop (or peek) *outStack*'s top.

Why pouring only when *outStack* is empty gives amortized $O(1)$

If we poured on *every* dequeue, each dequeue would cost $O(n)$. Instead, we only pour when *outStack* is completely empty – and once poured, *outStack* correctly holds a whole batch of elements in the right FIFO order, which can each be dequeued in $O(1)$ without needing another pour until that batch is exhausted. Each individual element is poured (moved from *inStack* to *outStack*) at most *once* in its entire lifetime, so across any sequence of n enqueue/dequeue operations, the total cost of all pours combined is $O(n)$, giving $O(1)$ *amortized* time per operation even though some individual dequeues are more expensive than others.

Algorithm

1. *enqueue(x)*: push x onto *inStack*.
2. *dequeue()* / *front()*: if *outStack* is empty, repeatedly pop from *inStack* and push onto *outStack* until *inStack* is empty. Then pop (or peek) *outStack*.

Dry run

Dry Run. Enqueue 1, 2, 3; dequeue twice; enqueue 4; dequeue once.

- Enqueue 1, 2, 3: *inStack* (bottom to top) = [1, 2, 3]; *outStack* = [].
- Dequeue: *outStack* empty, so pour: pop 3, 2, 1 from *inStack*, push onto *outStack*, giving *outStack* (bottom to top) = [3, 2, 1]. Pop *outStack*'s top: returns 1. *outStack* = [3, 2].
- Dequeue: *outStack* not empty ([3, 2]). Pop top: returns 2. *outStack* = [3].
- Enqueue 4: push onto *inStack* = [4]. *outStack* still = [3], untouched.
- Dequeue: *outStack* not empty. Pop top: returns 3.

The dequeue sequence 1, 2, 3 correctly matches the enqueue order.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class QueueUsingStack {
5     stack<int> inStack, outStack;
6
7     void transferIfNeeded() {
8         if (outStack.empty()) {
9             while (!inStack.empty()) {
10                 outStack.push(inStack.top());
11                 inStack.pop();
12             }
13         }
14     }
15
16 public:
17     void enqueue(int x) {
18         inStack.push(x);
19     }
20
21     int dequeue() {
22         transferIfNeeded();
23         int val = outStack.top();
24         outStack.pop();
25         return val;
26     }
27
28     int front() {
29         transferIfNeeded();
30         return outStack.top();
31     }
32 };
33
34 int main() {
35     QueueUsingStack q;
36     q.enqueue(1); q.enqueue(2); q.enqueue(3);
37     cout << q.dequeue() << " " << q.dequeue() << endl;
38     q.enqueue(4);
39     cout << q.dequeue() << endl;
40     return 0;
41 }

```

Complexity Analysis

Time Complexity. *enqueue* is always $O(1)$. *dequeue* and *front* are $O(1)$ **amortized** (occasionally $O(n)$ for a pour, but each element is poured at most once across its lifetime).
Space Complexity. $O(n)$ across the two stacks combined.

Key Takeaway

Reversing twice restores original order – this is the entire idea behind using two stacks to simulate a queue, and it is worth remembering as a general trick: any time an algorithm needs to “undo” a reversal it was forced into, pouring through a second stack is often the simplest fix.

1.7 Min Stack

Problem Statement. Design a stack supporting $push(x)$, $pop()$, $top()$, and $getMin()$ – the last returning the minimum element currently in the stack – all in $O(1)$ time.

Pattern Recognition

“Maintain a running aggregate (min , max , sum) that must stay correct as elements are pushed and popped” is the keyword pattern here. A plain running variable breaks the moment a pop removes the element that was responsible for the current minimum – there would be no way to recover the *previous* minimum without re-scanning. The fix is to keep a full **history** of the minimum, one entry per stack position, so that popping the main stack and popping the history together always keeps both in sync.

Understanding the problem carefully

The difficulty is specifically about pop : tracking the minimum as elements are pushed is trivial (compare the new element to the current minimum). The problem is that after a pop , the minimum might need to *revert* to whatever it was before the popped element was pushed – and that historical value must be available instantly, not recomputed.

A Second Stack Tracks the Minimum-So-Far at Every Depth

Maintain an auxiliary *minStack*, parallel to the main stack. On $push(x)$: push x onto the main stack as usual, and push $\min(x, \text{minStack.top}())$ onto *minStack* (or just x if *minStack* is currently empty). On $pop()$: pop both stacks together. $getMin()$ simply reads *minStack*’s top.

Why popping both stacks together keeps the invariant correct

minStack’s i -th entry (counting from the bottom) is, by construction, the minimum of the main stack’s bottom i elements at the moment the i -th element was pushed – and since both stacks always have the same height and are pushed to and popped from in lockstep, *minStack*’s current top is always exactly the minimum of *whatever the main stack currently contains*, not some stale value from before a pop. This is the same “parallel history” idea that makes undo/redo systems work: rather than computing an aggregate forward only, store enough history to reconstruct any previous state instantly.

Algorithm

1. $push(x)$: push x onto *mainStack*. Push $\min(x, \text{current } getMin())$ onto *minStack* (using x itself if *minStack* was empty).
2. $pop()$: pop both *mainStack* and *minStack*.
3. $top()$: return *mainStack*’s top.
4. $getMin()$: return *minStack*’s top.

Dry run

Dry Run. Push 5, 3, 7, 2; pop once; query $getMin$.

- Push 5: $\text{main} = [5]$, $\text{min} = [5]$.

- Push 3: $\text{main} = [5, 3]$, $\text{min} = [5, \min(3, 5)=3]$.
- Push 7: $\text{main} = [5, 3, 7]$, $\text{min} = [5, 3, \min(7, 3)=3]$.
- Push 2: $\text{main} = [5, 3, 7, 2]$, $\text{min} = [5, 3, 3, \min(2, 3)=2]$.
- Pop: both stacks lose their top. $\text{main} = [5, 3, 7]$, $\text{min} = [5, 3, 3]$.
- `getMin()`: top of min stack is 3.

Correct: after removing 2, the remaining stack $[5, 3, 7]$ has minimum 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class MinStack {
5     stack<int> mainStack, minStack;
6
7 public:
8     void push(int x) {
9         mainStack.push(x);
10        if (minStack.empty()) minStack.push(x);
11        else minStack.push(min(x, minStack.top()));
12    }
13
14    void pop() {
15        mainStack.pop();
16        minStack.pop();
17    }
18
19    int top() {
20        return mainStack.top();
21    }
22
23    int getMin() {
24        return minStack.top();
25    }
26 };
27
28 int main() {
29     MinStack s;
30     s.push(5); s.push(3); s.push(7); s.push(2);
31     s.pop();
32     cout << "Min: " << s.getMin() << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. $O(1)$ for every operation. **Space Complexity.** $O(n)$, doubled from a plain stack since every push adds one entry to *both* stacks. This can be reduced to $O(1)$ *extra* space (still $O(n)$ total, but without a second full stack) using a trick where only the previous minimum is pushed onto the main stack itself when a new minimum is encountered, encoded so it can be distinguished and restored on pop – a worthwhile optimization to know exists, though the two-stack version above is the version to derive and explain first.

Key Takeaway

Whenever an aggregate (min, max, or anything else that a plain running variable cannot correctly *revert* after a removal) must stay valid across both insertions and removals, keep a parallel history stack rather than trying to recompute the aggregate from scratch on every pop.

1.8 Valid Parentheses

Problem Statement. Given a string s consisting only of the characters '(', ')', '[', ']', '{', '}', determine whether the brackets are **valid**: every closing bracket must match the most recently opened, still-unmatched bracket of the same type, and every opening bracket must eventually be closed.

Pattern Recognition

“*Matching/nesting*” is the most direct stack keyword there is: the defining feature of valid nesting is that the bracket needing to be closed **next** is always the **most recently opened** one still unmatched – exactly LIFO order. Whenever a problem describes validity in terms of nested or paired structures (brackets, tags, nested function calls), a stack of “currently open, unmatched” items is almost always the right tool.

Understanding the problem carefully

Scan the string left to right. Every opening bracket is a commitment that must be fulfilled later, in reverse order of when the commitments were made – the bracket opened most recently must be the first one closed, because brackets cannot cross (that would not be “nesting”). Pushing each opening bracket onto a stack, and popping when we see a closing bracket, mirrors this exactly: the stack’s top is always the innermost currently-open bracket, which is precisely the one any new closing bracket must match.

Stack of Open Brackets

For each character: if it is an opening bracket, push it. If it is a closing bracket: the stack must be non-empty, and its top must be the *matching* opening bracket; if either check fails, the string is invalid. Otherwise, pop the stack (this pair is now resolved). After scanning the whole string, the string is valid if and only if the stack is empty (every opened bracket was eventually closed).

Why both checks (stack non-empty, and top matches) are necessary

If the stack is empty when a closing bracket appears, there is no open bracket left to match it against – the closing bracket is unmatched, and the string is invalid (e.g. `)` alone). If the stack’s top does not match the closing bracket’s type (e.g. the top is `'('` but we just saw `']'`), the brackets have crossed incorrectly (e.g. `"([")`), which also invalidates the string. And even if every individual closing bracket finds a match, any *leftover* opening brackets at the end (a non-empty stack) mean some opens were never closed at all (e.g. `"(("`, missing both closing parentheses) – this is why the final emptiness check is also required, not just the per-character checks.

Algorithm

1. Initialize an empty stack.
2. For each character c in s :
 - (a) If c is an opening bracket: push c .
 - (b) Else (c is a closing bracket): if the stack is empty, or its top is not the matching opening bracket for c , return **false**. Otherwise, pop the stack.
3. Return whether the stack is now empty.

Algorithm 1 Valid Parentheses

```

1: procedure ISVALID( $s$ )
2:    $stack \leftarrow$  empty
3:   for each character  $c$  in  $s$  do
4:     if  $c \in \{ (, [, \{ \}$  then
5:       push  $c$  onto  $stack$ 
6:     else
7:       if  $stack$  is empty or top of  $stack$  does not match  $c$  then
8:         return false
9:       end if
10:      pop  $stack$ 
11:    end if
12:  end for
13:  return ( $stack$  is empty)
14: end procedure

```

Dry run

Dry Run. Take $s = "\{ [()] \}"$.

- '{': push. Stack = [{].
- '[': push. Stack = [{, [].
- '(': push. Stack = [{, [, (].
- ')': top is '(', matches. Pop. Stack = [{, [].
- ']': top is '[', matches. Pop. Stack = [{].
- '}': top is '{', matches. Pop. Stack = [].

Stack is empty at the end: **true**.

Now take $s = "[]"$: push '['; then see ']', top is '[', which does not match ']' – return **false** immediately.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isValid(string s) {
5     stack<char> st;
6     for (char c : s) {
7         if (c == '(' || c == '[' || c == '{') {
8             st.push(c);
9         } else {
10             if (st.empty()) return false;
11             char top = st.top();
12             if ((c == ')' && top != '(') ||
13                 (c == ']' && top != '[') ||
14                 (c == '}' && top != '{')) {
15                 return false;
16             }
17             st.pop();
18         }
19     }
20     return st.empty();
21 }
22
23 int main() {
24     cout << (isValid("[()]{}") ? "true" : "false") << endl;
25     cout << (isValid("[]") ? "true" : "false") << endl;
26     return 0;
27 }

```

Complexity Analysis

Time Complexity. Each character is processed once with constant work, giving

$$O(n).$$

Space Complexity. $O(n)$ in the worst case (a string of all opening brackets pushes every character onto the stack).

Key Takeaway

“The next thing that needs to be resolved is always the most recently opened, still-unresolved thing” is the defining signature of a stack problem. This is not limited to brackets – the same template applies to validating nested tags, matching function call/return pairs, or undoing actions in reverse order of when they were performed.

1.9 Infix to Postfix Conversion

We now turn to a group of six closely related problems: converting between the three ways of writing an arithmetic expression. **Infix** places operators between operands ($a + b$) and needs parentheses and precedence rules to remove ambiguity. **Postfix** (operators after their operands, $ab+$) and **prefix** (operators before, $+ab$) are both unambiguous without any parentheses, which is exactly why calculators and compilers prefer them internally – and why a stack-based scan can evaluate or convert them in a single pass.

Problem Statement. Given an infix expression (operands, the binary operators $+ - * / \wedge$, and parentheses), convert it to postfix notation.

Pattern Recognition

“Convert an expression respecting operator precedence and parentheses” is the Shunting-Yard algorithm’s signature: a stack holds operators *waiting* to be placed into the output, and the central decision at every operator is “do any waiting operators have **equal or higher** precedence than the one I just read, and if so, should they come out first?” – exactly the kind of ordering decision a stack resolves naturally.

Understanding the problem carefully

In postfix, an operator appears *immediately after* both of its operands are fully written – which means an operator can only be emitted to the output once we are certain nothing of higher (or equal, for left-associative operators) precedence is still waiting to bind tighter around it. Parentheses simply force this resolution explicitly: everything inside a (...) must be fully resolved into postfix before the parenthesis is allowed to close.

Precedence and Associativity

Assign precedence: $\wedge$ (highest, right-associative) $>$ $*$ / (left-associative) $>$ $+$ $-$ (lowest, left-associative). When deciding whether to pop an operator already on the stack before pushing a new one: pop if the stacked operator has **strictly higher** precedence, or **equal** precedence *and* the new operator is left-associative (for a right-associative operator like $\wedge$, equal precedence does *not* trigger a pop, since $\wedge$ groups right-to-left: $2\wedge 3\wedge 2$ means $2\wedge (3\wedge 2)$, not $(2\wedge 3)\wedge 2$).

Why popping by precedence produces correct postfix

An operator should only be written to postfix output once everything that needs to bind *more tightly* around its operands has already been resolved. If the stack’s top has strictly higher precedence, it must finish binding its own operands first (it is “more urgent”), so we pop it to the output before pushing the new, lower-precedence operator. If precedences are equal and the operator is left-associative, the earlier (already-stacked) operator should also be resolved first, since left-associativity means we group left to right – e.g. $a - b - c$ means $(a - b) - c$, so the first $-$ must close over a, b before the second $-$ is considered. Right-associative operators reverse this rule specifically to preserve right-to-left grouping.

Algorithm

1. Scan left to right. For each token:
 - (a) Operand: append to output.
 - (b) ‘(’: push.
 - (c) ‘)’’: pop and append to output until ‘(’ is popped (discard both parentheses).
 - (d) Operator op : while the stack’s top is an operator with higher precedence, or equal precedence with op left-associative: pop and append to output. Then push op .
2. After the scan, pop all remaining operators to the output.

Algorithm 2 Infix to Postfix

```

1: procedure INFIXTOPOSTFIX(s)
2:   output  $\leftarrow$  empty, stack  $\leftarrow$  empty
3:   for each token t in s do
4:     if t is an operand then append t to output
5:     else if t = ( then push t
6:     else if t = ) then
7:       while top  $\neq$  ( do pop, append to output
8:       end while
9:       pop the (
10:    else
11:      while stack not empty and top has higher precedence, or (equal precedence and
        t is left-assoc) do
12:        pop, append to output
13:      end while
14:      push t
15:    end if
16:  end for
17:  while stack not empty do pop, append to output
18:  end while
19:  return output
20: end procedure

```

Dry run

Dry Run. Take $s = "a+b*c"$.

- a: output = "a".
- +: stack empty, push. Stack = [+].
- b: output = "ab".
- *: top is + (lower precedence than *), no pop. Push. Stack = [+,*].
- c: output = "abc".
- End of scan: pop all. Pop *: output = "abc*". Pop +: output = "abc**".

Result: "abc**", correctly reflecting that * binds tighter than + (i.e. $a + (b * c)$).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int precedence(char op) {
5     if (op == '^') return 3;
6     if (op == '*' || op == '/') return 2;
7     if (op == '+' || op == '-') return 1;
8     return 0;
9 }
10
11 bool isRightAssoc(char op) { return op == '^'; }

```

```

12
13 string infixToPostfix(string s) {
14     string output;
15     stack<char> st;
16
17     for (char c : s) {
18         if (isalnum(c)) {
19             output += c;
20         } else if (c == '(') {
21             st.push(c);
22         } else if (c == ')') {
23             while (st.top() != '(') {
24                 output += st.top();
25                 st.pop();
26             }
27             st.pop(); // remove '('
28         } else { // operator
29             while (!st.empty() && st.top() != '(' &&
30                 (precedence(st.top()) > precedence(c) ||
31                 (precedence(st.top()) == precedence(c) && !
32                     isRightAssoc(c)))) {
33                 output += st.top();
34                 st.pop();
35             }
36             st.push(c);
37         }
38     }
39     while (!st.empty()) {
40         output += st.top();
41         st.pop();
42     }
43     return output;
44 }
45
46 int main() {
47     cout << infixToPostfix("a+b*c") << endl;
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. Each token is pushed and popped at most once, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack and output in the worst case.

Key Takeaway

The precedence-and-associativity comparison rule derived here is reused, completely unchanged, in Infix to Prefix (Section 1.10) – only the direction of scanning and the meaning of the output change.

1.10 Infix to Prefix Conversion

Problem Statement. Given an infix expression, convert it to prefix notation.

Pattern Recognition

“*Convert to prefix*” immediately after having just solved “convert to postfix” (Section 1.9) is an invitation to **reuse** that algorithm via a symmetry trick, rather than deriving a new one: prefix is postfix *read backward*, with operators and operands swapped in scanning direction – so reversing the input, adapting the tie-breaking rule, running the same postfix algorithm, and reversing the output again gets us there with no new derivation.

Understanding the problem carefully

If postfix writes operators *after* their operands, prefix writes them *before* – which is exactly what you get by mirror-imaging a postfix expression (reversing it character by character) *and* reversing each multi-character operand if there were any. So the natural strategy is: reverse the input expression (also swapping every ‘(’ with ‘)’ and vice versa, since reversal flips which parenthesis opens and which closes), run essentially the same Shunting-Yard scan as Section 1.9 on this reversed-and-swapped string, then reverse the resulting output once more to undo the initial reversal.

The One Rule That Must Change

When running the postfix algorithm on the reversed string, the tie-break rule for **equal precedence** must flip: do *not* pop an equal-precedence operator already on the stack before pushing a new one, *regardless of true associativity*. This is because reversing the scan direction also reverses which operand of a left-associative chain we encounter first; treating every tie as if it were right-associative during this reversed scan is what correctly preserves the original left-to-right grouping once everything is reversed back.

Why reverse-scan-reverse is correct

This is a direct application of the same “reverse twice to flip an order without breaking internal structure” idea seen in Section 1.6. Reversing the infix string converts a left-to-right scan that would naturally produce postfix into one that, after the tie-break adjustment, produces the mirror image of postfix – which, when reversed back to normal reading order, is exactly prefix. The parenthesis-swapping step is necessary because reversing a string turns every originally-opening parenthesis into a closing one (textually) and vice versa, and the algorithm needs them to still behave correctly as openers and closers during the reversed scan.

Algorithm

1. Reverse s ; swap every ‘(’ with ‘)’ and vice versa.
2. Run the Shunting-Yard postfix algorithm (Algorithm 1.9) on this modified string, with the tie-break rule changed to *never* pop on equal precedence.
3. Reverse the resulting output string. This is the prefix expression.

Dry run

Dry Run. Take $s = \text{"a+b*c"}$.

- Reverse and swap brackets (no brackets here): "c*b+a" .
- Run the postfix algorithm (never popping on ties):
 - c : output = "c" .

- *: push. Stack = [*].
- b: output = "cb".
- +: top is * (higher precedence), pop: output = "cb*". Push +. Stack = [+].
- a: output = "cb*a".
- End: pop +: output = "cb*a+".

- Reverse the output: "+a*bc".

Result: "+a*bc", correctly representing $a + (b * c)$ in prefix.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int precedence(char op) {
5     if (op == '^') return 3;
6     if (op == '*' || op == '/') return 2;
7     if (op == '+' || op == '-') return 1;
8     return 0;
9 }
10
11 string infixToPrefix(string s) {
12     reverse(s.begin(), s.end());
13     for (char& c : s) {
14         if (c == '(') c = ')';
15         else if (c == ')') c = '(';
16     }
17
18     string output;
19     stack<char> st;
20
21     for (char c : s) {
22         if (isalnum(c)) {
23             output += c;
24         } else if (c == '(') {
25             st.push(c);
26         } else if (c == ')') {
27             while (st.top() != '(') {
28                 output += st.top();
29                 st.pop();
30             }
31             st.pop();
32         } else { // operator: never pop on equal precedence
33             while (!st.empty() && st.top() != '(' &&
34                 precedence(st.top()) > precedence(c)) {
35                 output += st.top();
36                 st.pop();
37             }
38             st.push(c);
39         }
40     }
41     while (!st.empty()) {
42         output += st.top();
43         st.pop();
44     }

```

```

45     reverse(output.begin(), output.end());
46     return output;
47 }
48
49
50 int main() {
51     cout << infixToPrefix("a+b*c") << endl;
52     return 0;
53 }

```

Complexity Analysis

Time Complexity. $O(n)$, identical in structure to Section 1.9. **Space Complexity.** $O(n)$.

Key Takeaway

“Reverse, adapt the tie-break rule, run the algorithm you already have, reverse again” is a powerful general technique for converting a left-to-right algorithm into its mirror-image counterpart, without re-deriving the core logic. Recognize when a new problem is the *mirror image* of one you have already solved before writing new code from scratch.

1.11 Postfix to Infix Conversion

Problem Statement. Given a postfix expression, convert it to infix notation (fully parenthesized, to remove any ambiguity).

Pattern Recognition

Converting *from* postfix or prefix is structurally easier than converting *from* infix (Sections 1.9–1.10), because postfix and prefix already encode the expression tree’s structure unambiguously through ordering alone – there is no precedence or associativity to resolve. The keyword shift to notice: when the *source* notation is postfix or prefix, the stack holds **partially built sub-expressions** (strings), not raw operators, and every operator immediately and unambiguously tells you which two sub-expressions to combine.

Understanding the problem carefully

In postfix, by the time we encounter an operator while scanning left to right, both of its operands have already been fully read and are sitting on top of the stack (the most recent two pushes) – there is never any ambiguity about which sub-expressions an operator applies to, unlike infix where precedence must be consulted.

Stack of Sub-expressions

Scan left to right. Push every operand as a (string) sub-expression. On an operator: pop the top two sub-expressions – call the first popped *right* and the second popped *left* (since *left* was pushed earlier and so sits one level deeper). Combine them as (*left op right*), fully parenthesized, and push this combined string back as a single sub-expression.

Why the most recent two pushes are always the correct operands

Postfix notation guarantees that an operator always immediately follows the complete sub-expressions it operates on – this is precisely what “postfix” (operator *after* operands) means.

So whatever is on top of the stack at the moment we read an operator must be its right operand (read most recently), and whatever is just below that must be its left operand, with no possibility of some other, older sub-expression interfering, because every operator so far has already fully consumed and replaced its own operands before we reach this point.

Algorithm

1. Initialize an empty stack of strings.
2. For each token in the postfix expression:
 - (a) Operand: push it as a one-character string.
 - (b) Operator *op*: pop *right*, pop *left*; push (*left op right*).
3. The stack's single remaining element is the infix result.

Dry run

Dry Run. Take "ab+c*".

- a: push. Stack = [a].
- b: push. Stack = [a, b].
- +: pop right= b, pop left= a. Push (a+b). Stack = [(a+b)].
- c: push. Stack = [(a+b), c].
- *: pop right= c, pop left= (a+b). Push ((a+b)*c).

Result: ((a+b)*c).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string postfixToInfix(string s) {
5     stack<string> st;
6
7     for (char c : s) {
8         if (isalnum(c)) {
9             st.push(string(1, c));
10        } else {
11            string right = st.top(); st.pop();
12            string left = st.top(); st.pop();
13            st.push("(" + left + string(1, c) + right + ")");
14        }
15    }
16    return st.top();
17 }
18
19 int main() {
20     cout << postfixToInfix("ab+c*") << endl;
21     return 0;
22 }
```

Complexity Analysis

Time Complexity. Each token triggers $O(1)$ stack operations, but string concatenation can cost up to $O(n)$ per operator in the worst case (building progressively longer strings), giving an overall worst case of

$$O(n^2)$$

for naive string concatenation (this can be improved to $O(n)$ with more careful string-building techniques, but $O(n^2)$ is what the straightforward implementation above gives).

Space Complexity. $O(n)$ for the stack of sub-expressions.

Key Takeaway

When converting *from* postfix or prefix, the stack holds sub-expressions (strings), not bare operators – every operator encountered has an unambiguous, immediate pair of operands to combine, with no precedence comparison ever needed. This is structurally simpler than converting *from* infix, and the same “stack of sub-expressions” idea reused, with only the combination format (parenthesized infix here, vs. prefix or postfix in the next few sections) changing.

1.12 Postfix to Prefix Conversion

Problem Statement. Given a postfix expression, convert it to prefix notation.

Pattern Recognition

Identical setup to Section 1.11 – the only change is *how* two popped sub-expressions are combined: instead of (*left op right*), prefix writes the operator **first**, with no parentheses needed at all (prefix, like postfix, is unambiguous without them).

Understanding the problem carefully

The stack-of-sub-expressions technique from Section 1.11 carries over completely unchanged; only the string-building step at each operator differs.

Combine as Prefix Instead of Infix

On an operator *op*: pop *right*, pop *left* (same order as before); push *op+left+right* (operator first, then both sub-expressions back-to-back, no parentheses or spaces needed since prefix length is self-delimiting when operands are single characters, or separated by a delimiter for multi-character operands).

Algorithm

1. Initialize an empty stack of strings.
2. For each token: operand $\rightarrow$ push as a string. Operator *op* $\rightarrow$ pop *right*, pop *left*; push *op+left+right*.
3. The stack’s single remaining element is the prefix result.

Dry run

Dry Run. Take "ab+c*".

- a: push. Stack = [a].
- b: push. Stack = [a, b].
- +: pop right= b, left= a. Push +ab. Stack = [+ab].
- c: push. Stack = [+ab, c].
- *: pop right= c, left= +ab. Push **abc.

Result: **abc, correctly representing $(a + b) * c$ in prefix.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string postfixToPrefix(string s) {
5     stack<string> st;
6
7     for (char c : s) {
8         if (isalnum(c)) {
9             st.push(string(1, c));
10        } else {
11            string right = st.top(); st.pop();
12            string left = st.top(); st.pop();
13            st.push(string(1, c) + left + right);
14        }
15    }
16    return st.top();
17 }
18
19 int main() {
20     cout << postfixToPrefix("ab+c*") << endl;
21     return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(n^2)$ worst case with naive string concatenation, as in Section 1.11.

Space Complexity. $O(n)$.

Key Takeaway

This section is intentionally a near-copy of Section 1.11 – the lesson is that once you have the “stack of sub-expressions, combine on every operator” skeleton right, every output *format* (parenthesized infix, prefix, or postfix-of-a-different-source) is just a one-line change to how the combination step builds its string.

1.13 Prefix to Infix Conversion

Problem Statement. Given a prefix expression, convert it to infix notation (fully parenthesized).

Pattern Recognition

Prefix writes the operator *before* its operands, so by the time we encounter an operator while scanning **left to right**, its operands have not been read yet – the stack-of-sub-expressions trick from Sections 1.11–1.12 only works smoothly when operands are already available *before* their operator is seen. The fix: scan **right to left** instead, which makes prefix behave exactly like postfix did under a left-to-right scan.

Understanding the problem carefully

Reading a prefix expression from right to left, an operand is encountered *before* the operator that uses it (mirroring how, in postfix read left to right, an operand is also encountered before its operator) – so the same stack-of-sub-expressions technique applies, just with the scan direction reversed.

Scan Right to Left, Same Combination Rule

Scan from the last character to the first. Push every operand. On an operator *op*: pop the top sub-expression as *left* (it was pushed most recently, and since we are scanning backward, “most recent” corresponds to the operand that appears *first*, i.e. leftmost, after this operator when read normally); pop the next as *right*. Push (*left op right*).

Why the pop order flips relative to postfix

In postfix, scanning left to right, the operand pushed *last* (closest to the operator, on its left in the text) is the **right** operand semantically (e.g. in **ab+**, *b* is pushed right before **+**, and *a + b* has *b* as the right operand). In prefix, scanning *right to left*, the operand encountered (and pushed) most recently before the operator is the one immediately to the operator’s right in the original text – which, in prefix’s *op A B* structure, is *A*, the **left** operand. So the pop order is mirrored relative to postfix’s, which is exactly why *left* is popped first here.

Algorithm

1. Initialize an empty stack of strings.
2. For each token, scanning from right to left:
 - (a) Operand: push it.
 - (b) Operator *op*: pop *left*, pop *right*; push (*left op right*).
3. The stack’s single remaining element is the infix result.

Dry run

Dry Run. Take **"*+abc"** (representing $(a + b) * c$).

Scanning right to left: **c, b, a, +, ***.

- **c**: push. Stack = [**c**].
- **b**: push. Stack = [**c, b**].
- **a**: push. Stack = [**c, b, a**].
- **+**: pop left = **a**, pop right = **b**. Push (**a+b**). Stack = [**c, (a+b)**].

- *: pop left= (a+b), pop right= c. Push ((a+b)*c).

Result: ((a+b)*c) – matching Section 1.11’s result for the same underlying expression.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string prefixToInfix(string s) {
5     stack<string> st;
6
7     for (int i = s.size() - 1; i >= 0; i--) {
8         char c = s[i];
9         if (isalnum(c)) {
10             st.push(string(1, c));
11         } else {
12             string left = st.top(); st.pop();
13             string right = st.top(); st.pop();
14             st.push("(" + left + string(1, c) + right + ")");
15         }
16     }
17     return st.top();
18 }
19
20 int main() {
21     cout << prefixToInfix(".*+abc") << endl;
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ worst case with naive string concatenation, $O(n)$ with more careful string building. **Space Complexity.** $O(n)$.

Key Takeaway

Scanning right to left is to prefix what scanning left to right is to postfix – both make operands available before the operator that needs them. Whenever a problem’s natural scan direction would otherwise present an operator before its operands are ready, check whether reversing the scan direction restores the same simple stack technique.

1.14 Prefix to Postfix Conversion

Problem Statement. Given a prefix expression, convert it to postfix notation.

Pattern Recognition

The last of the six conversions, and by now a pure exercise in combining two already-derived ideas: scan **right to left** (as in Section 1.13, since the source is prefix), and combine each pair as *left + right + op* (as in Section 1.12’s combination style, just operator *last* instead of first, and no parentheses needed).

Understanding the problem carefully

No new idea is required: take the right-to-left scan direction from Section 1.13 (because the source is prefix) and the parenthesis-free, operator-position-only combination style appropriate for postfix output (operator written last).

Combine as Postfix Instead of Infix

Scan right to left. Push every operand. On operator *op*: pop *left*, pop *right* (same pop order as Section 1.13, since the source notation and scan direction are unchanged); push *left + right + op*.

Algorithm

1. Initialize an empty stack of strings.
2. For each token, scanning right to left: operand $\rightarrow$ push. Operator *op* $\rightarrow$ pop *left*, pop *right*; push *left + right + op*.
3. The stack's single remaining element is the postfix result.

Dry run

Dry Run. Take "**+abc*".

Scanning right to left: *c*, *b*, *a*, *+*, ***.

- *c*: push. Stack = [*c*].
- *b*: push. Stack = [*c*, *b*].
- *a*: push. Stack = [*c*, *b*, *a*].
- *+*: pop *left*= *a*, pop *right*= *b*. Push *ab+*. Stack = [*c*, *ab+*].
- ***: pop *left*= *ab+*, pop *right*= *c*. Push *ab+c**.

Result: *ab+c** – exactly the postfix expression we started from back in Section 1.11, confirming the whole six-conversion group is internally consistent.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string prefixToPostfix(string s) {
5     stack<string> st;
6
7     for (int i = s.size() - 1; i >= 0; i--) {
8         char c = s[i];
9         if (isalnum(c)) {
10             st.push(string(1, c));
11         } else {
12             string left = st.top(); st.pop();
13             string right = st.top(); st.pop();
14             st.push(left + right + string(1, c));
15         }
16     }

```

```

17     return st.top();
18 }
19
20 int main() {
21     cout << prefixToPostfix("++abc") << endl;
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ worst case with naive string concatenation, $O(n)$ with more careful string building. **Space Complexity.** $O(n)$.

Key Takeaway

Across all six conversions in this group, there are really only two independent decisions: *which direction to scan* (left to right if the source is postfix or plain infix; right to left if the source is prefix) and *how to format the combination step* (parenthesized infix, operator-first prefix, or operator-last postfix). Every one of the six problems is a different combination of these same two choices.

1.15 Next Greater Element I

We now turn to the second major group of problems in this chapter: the **monotonic stack**, introduced in Section 1.1. This first problem is the purest possible instance of the pattern, and every later monotonic-stack problem in this chapter is a variation or extension of the technique derived here.

Problem Statement. We are given two arrays, *nums1* and *nums2*, both containing distinct integers, where every element of *nums1* is also present somewhere in *nums2*. For each element of *nums1*, find its **next greater element** in *nums2*: the first element to its right, within *nums2*, that is strictly greater than it. If no such element exists, the answer for that element is -1 .

Pattern Recognition

“Next greater element” is, almost word for word, the keyword from Section 1.1’s pattern list. The structure here is two-part: first solve “next greater element, for every position, within *nums2*” (a self-contained monotonic-stack problem), then a simple hash-map lookup translates that solved array into answers for *nums1*.

Understanding the problem carefully

The core sub-problem is: for every index i in *nums2*, find the nearest index $j > i$ with $nums2[j] > nums2[i]$, or report none exists. A brute-force solution checks every pair, costing $O(n^2)$. The monotonic stack from Section 1.1 solves this in $O(n)$ by scanning right to left and maintaining a stack that is always strictly decreasing from bottom to top.

Monotonic Decreasing Stack, Scanned Right to Left

Scan *nums2* from right to left. Before processing $nums2[i]$, pop every stack element $\leq nums2[i]$ (they can never be the answer for anything still to the left, since $nums2[i]$ is closer and at least as large). Whatever remains on top of the stack after this popping (if anything) is exactly i ’s next greater element. Then push $nums2[i]$.

Why this gives the correct next greater element for every index

After popping every $\leq \text{nums2}[i]$ value, the stack's remaining top – if any – is the *nearest* value to the right of i that exceeds $\text{nums2}[i]$. This holds because the stack only ever contains values in decreasing order from bottom to top, and every value currently on the stack represents some index to the right of i that has not yet been “defeated” by anything closer and at least as large; the topmost (most recently pushed, hence nearest) surviving value is therefore guaranteed to be the closest qualifying greater value. Once computed for every index of nums2 , store the results in a hash map keyed by value (values are distinct, so this is unambiguous), and answer each query for nums1 with a single $O(1)$ lookup.

Algorithm

1. Initialize an empty stack and an empty hash map *nextGreater*.
2. For i from $n - 1$ down to 0 (scanning nums2 right to left):
 - (a) While the stack is not empty and its top $\leq \text{nums2}[i]$: pop.
 - (b) $\text{nextGreater}[\text{nums2}[i]] \leftarrow$ the stack's top if non-empty, else -1 .
 - (c) Push $\text{nums2}[i]$.
3. For each query value in nums1 : look up $\text{nextGreater}[\text{value}]$.

Algorithm 3 Next Greater Element I

```

1: procedure NEXTGREATERELEMENT( $\text{nums1}, \text{nums2}$ )
2:    $\text{stack} \leftarrow$  empty,  $\text{nextGreater} \leftarrow$  empty map
3:   for  $i \leftarrow n - 1$  down to 0 do
4:     while  $\text{stack}$  not empty and top  $\leq \text{nums2}[i]$  do
5:       pop
6:     end while
7:      $\text{nextGreater}[\text{nums2}[i]] \leftarrow (\text{stack empty} ? -1 : \text{top})$ 
8:     push  $\text{nums2}[i]$ 
9:   end for
10:  return [ $\text{nextGreater}[x]$  for each  $x$  in  $\text{nums1}$ ]
11: end procedure

```

Dry run

Dry Run. Take $\text{nums2} = [1, 3, 4, 2]$, $\text{nums1} = [4, 1, 2]$.

Scanning right to left:

- $i = 3$ (value 2): stack empty. $\text{nextGreater}[2] = -1$. Push 2. Stack = [2].
- $i = 2$ (value 4): top is $2 \leq 4$, pop. Stack empty. $\text{nextGreater}[4] = -1$. Push 4. Stack = [4].
- $i = 1$ (value 3): top is $4 > 3$, no pop. $\text{nextGreater}[3] = 4$. Push 3. Stack = [4, 3].
- $i = 0$ (value 1): top is $3 > 1$, no pop. $\text{nextGreater}[1] = 3$. Push 1. Stack = [4, 3, 1].

Map: $\{2:-1, 4:-1, 3:4, 1:3\}$. Querying $\text{nums1} = [4, 1, 2]$: answers $[-1, 3, -1]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2)
5 {
6     unordered_map<int,int> nextGreater;
7     stack<int> st;
8
9     for (int i = nums2.size() - 1; i >= 0; i--) {
10         while (!st.empty() && st.top() <= nums2[i]) {
11             st.pop();
12         }
13         nextGreater[nums2[i]] = st.empty() ? -1 : st.top();
14         st.push(nums2[i]);
15     }
16
17     vector<int> result;
18     for (int x : nums1) result.push_back(nextGreater[x]);
19     return result;
20 }
21
22 int main() {
23     vector<int> nums2 = {1, 3, 4, 2};
24     vector<int> nums1 = {4, 1, 2};
25     for (int x : nextGreaterElement(nums1, nums2)) cout << x << " ";
26     cout << endl;
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. Each element of *nums2* is pushed once and popped at most once across the entire scan, giving $O(n)$ for the preprocessing; each of the m queries is an $O(1)$ map lookup, giving an overall time complexity of

$$O(n + m).$$

Space Complexity. $O(n)$ for the stack and hash map.

Key Takeaway

The monotonic stack derived here – scan right to left, pop everything $\leq$ the current element, the survivor on top is the answer, then push – is the master template for this entire sub-chapter. Every remaining monotonic-stack section either applies this template directly (next smaller, previous greater/smaller) or builds one more layer on top of it (histogram area, trapping rain water, subarray sums).

1.16 Next Greater Element II (Circular Array)

Problem Statement. We are given a **circular** array *nums* (the element after the last one wraps around to the first). For every element, find its next greater element, searching circularly – i.e. we may wrap around the end of the array once, but not more than once.

Pattern Recognition

The keyword “*circular*” attached to a next-greater-element problem signals that the search for each element’s answer may need to continue past the array’s literal end and wrap to the beginning – but only **once around**, not indefinitely. The standard trick for simulating one extra lap around a circular array without actually duplicating it in memory is to iterate over **twice** the indices (0 to $2n - 1$), using the modulo operation to map each iteration index back into the real array.

Understanding the problem carefully

If we simply ran the Section 1.15 algorithm once over *nums*, an element near the *end* of the array would never get a chance to find a next-greater candidate that happens to sit near the *beginning* – the algorithm would incorrectly report -1 for it. Conceptually, we want to pretend the array is duplicated end to end ($nums + nums$) and search within that doubled view, but stop considering circular wraparound after one full lap (we must not wrap twice, or an element could incorrectly become its own next greater element).

Simulate a Doubled Array via Modulo Indexing

Run the same right-to-left monotonic-stack scan as Section 1.15, but let i range from $2n - 1$ down to 0, accessing $nums[i \bmod n]$ at each step. Only *record* an answer for $i < n$ (the first lap, which corresponds to real array positions); the second lap ($i \geq n$, processed first since we scan from $2n - 1$ downward) exists purely to *seed the stack* with candidates that wrap around from the beginning of the array, without double-counting any element as its own answer.

Why processing the second lap first, without recording answers, is correct

Scanning from $2n - 1$ down to 0 means we process the conceptual “second copy” of the array (indices n to $2n - 1$, mapping to real indices 0 to $n - 1$) *before* the conceptual “first copy” (indices 0 to $n - 1$). By the time we reach real index i in the first-copy pass (i.e. when the loop variable itself is also i , having already processed $i + 1, \dots, 2n - 1$), the stack already reflects every element that could serve as i ’s next greater element either later in the same lap *or* wrapped around from the start of the array (since those wrapped candidates were pushed during the second-lap pass that ran first). This correctly captures one full circular lookahead without ever risking a second wraparound, since each real index contributes to the stack at most twice (once per lap) and answers are only recorded during the final, first-copy pass.

Algorithm

1. Initialize an empty stack and a result array of size n , filled with -1 .
2. For i from $2n - 1$ down to 0:
 - (a) Let $val \leftarrow nums[i \bmod n]$.
 - (b) While the stack is not empty and its top $\leq val$: pop.
 - (c) If $i < n$: set $result[i] \leftarrow$ the stack’s top if non-empty, else -1 .
 - (d) Push val .
3. Return *result*.

Algorithm 4 Next Greater Element II

```

1: procedure NEXTGREATERELEMENTSCIRCULAR(nums)
2:   result[0..n - 1]  $\leftarrow$  -1, stack  $\leftarrow$  empty
3:   for i  $\leftarrow$  2n - 1 down to 0 do
4:     val  $\leftarrow$  nums[i mod n]
5:     while stack not empty and top  $\leq$  val do
6:       pop
7:     end while
8:     if i < n and stack not empty then
9:       result[i]  $\leftarrow$  top
10:    end if
11:    push val
12:  end for
13:  return result
14: end procedure

```

Dry run

Dry Run. Take *nums* = [1, 2, 1] (*n* = 3).

- *i* = 5 (*val* = *nums*[2] = 1): *stack* empty. *i* $\geq$ *n*, no record. Push 1. *Stack* = [1].
- *i* = 4 (*val* = *nums*[1] = 2): top 1 $\leq$ 2, pop. *Stack* empty. *i* $\geq$ *n*, no record. Push 2. *Stack* = [2].
- *i* = 3 (*val* = *nums*[0] = 1): top 2 > 1, no pop. *i* $\geq$ *n*, no record. Push 1. *Stack* = [2, 1].
- *i* = 2 (*val* = *nums*[2] = 1): top 1 $\leq$ 1, pop. New top 2 > 1, no further pop. *i* < *n*: *result*[2] = 2. Push 1. *Stack* = [2, 1].
- *i* = 1 (*val* = *nums*[1] = 2): top 1 $\leq$ 2, pop. New top 2 $\leq$ 2, pop. *Stack* empty. *i* < *n*: *result*[1] = -1. Push 2. *Stack* = [2].
- *i* = 0 (*val* = *nums*[0] = 1): top 2 > 1, no pop. *i* < *n*: *result*[0] = 2. Push 1. *Stack* = [2, 1].

Result: [2, -1, 2]. Checking by hand: index 0 (value 1) wraps to find 2 at index 1; index 1 (value 2) has no greater element anywhere (it is the maximum); index 2 (value 1) wraps around to find 2 at index 1 (or directly via the doubled view, finds the 2 that appears right after it when wrapping).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> nextGreaterElementsCircular(vector<int>& nums) {
5     int n = nums.size();
6     vector<int> result(n, -1);
7     stack<int> st;
8
9     for (int i = 2 * n - 1; i >= 0; i--) {
10         int val = nums[i % n];
11         while (!st.empty() && st.top() <= val) {
12             st.pop();
13         }

```

```

14         if (i < n && !st.empty()) {
15             result[i] = st.top();
16         }
17         st.push(val);
18     }
19     return result;
20 }
21
22 int main() {
23     vector<int> nums = {1, 2, 1};
24     for (int x : nextGreaterElementsCircular(nums)) cout << x << " ";
25     cout << endl;
26     return 0;
27 }

```

Complexity Analysis

Time Complexity. The loop runs $2n$ iterations, each doing amortized $O(1)$ stack work (every element is pushed at most twice and popped at most twice across the whole run), giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack and result array.

Key Takeaway

“Circular array, search wrapping around once” has a standard, reusable fix: iterate over $2n$ virtual indices with modulo wraparound, using the first lap purely to seed state (here, the monotonic stack) and the second lap to record real answers. This avoids physically duplicating the array and generalizes to other circular-array problems beyond monotonic stacks.

1.17 Next Smaller Element

Problem Statement. Given an array arr , find, for every element, the next element to its right that is strictly **smaller**, or -1 if none exists.

Pattern Recognition

The mirror image of Section 1.15: flip every comparison, and the monotonic stack should now be kept strictly **increasing** from bottom to top instead of decreasing.

Understanding the problem carefully

By the same reasoning as next greater element, scanning right to left: before processing $arr[i]$, any stack element $\geq arr[i]$ can never be the “next smaller” answer for anything still to the left, since $arr[i]$ is closer and at least as small. Popping these and keeping whatever survives gives the nearest qualifying smaller value.

Monotonic Increasing Stack, Scanned Right to Left

Scan right to left. Before processing $arr[i]$, pop every stack element $\geq arr[i]$. The survivor on top (if any) is i ’s next smaller element. Push $arr[i]$.

Algorithm

1. Initialize an empty stack and a result array.
2. For i from $n - 1$ down to 0:
 - (a) While the stack is not empty and its top $\geq arr[i]$: pop.
 - (b) $result[i] \leftarrow$ the stack's top if non-empty, else -1 .
 - (c) Push $arr[i]$.
3. Return $result$.

Dry run

Dry Run. Take $arr = [4, 8, 5, 2, 25]$.

- $i = 4$ (25): stack empty. $result[4] = -1$. Push 25.
- $i = 3$ (2): top $25 \geq 2$, pop. Stack empty. $result[3] = -1$. Push 2.
- $i = 2$ (5): top $2 < 5$, no pop. $result[2] = 2$. Push 5. Stack = $[2, 5]$.
- $i = 1$ (8): top $5 < 8$, no pop. $result[1] = 5$. Push 8. Stack = $[2, 5, 8]$.
- $i = 0$ (4): top $8 \geq 4$, pop. New top $5 \geq 4$, pop. New top $2 < 4$, no pop. $result[0] = 2$. Push 4.

Result: $[2, 5, 2, -1, -1]$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> nextSmallerElement(vector<int>& arr) {
5      int n = arr.size();
6      vector<int> result(n, -1);
7      stack<int> st;
8
9      for (int i = n - 1; i >= 0; i--) {
10         while (!st.empty() && st.top() >= arr[i]) {
11             st.pop();
12         }
13         result[i] = st.empty() ? -1 : st.top();
14         st.push(arr[i]);
15     }
16     return result;
17 }
18
19 int main() {
20     vector<int> arr = {4, 8, 5, 2, 25};
21     for (int x : nextSmallerElement(arr)) cout << x << " ";
22     cout << endl;
23     return 0;
24 }
```

Complexity Analysis

Time Complexity. $O(n)$, by the same amortized argument as Section 1.15. **Space Complexity.** $O(n)$.

Key Takeaway

“Next/previous greater/smaller” is really one template with four dials: which direction to scan (left-to-right for *previous*, right-to-left for *next*) and which comparison the stack enforces (decreasing for *greater*, increasing for *smaller*). Once you have derived one of the four, the rest follow by flipping the relevant dial.

1.18 Number of NGEs to the Right

Problem Statement. Given an array arr of n elements and a list of query indices, for each query index i , count **how many** elements to the right of i (not just the nearest one) are strictly greater than $arr[i]$.

Pattern Recognition

This problem is grouped alongside the monotonic-stack problems because it shares their right-to-left scanning direction, but the keyword “*count how many*,” rather than “*find the nearest*,” is a signal that a plain monotonic stack is not quite the right tool: a stack only ever keeps a thin, decreasing/increasing *skyline* of values, not the full multiset of everything seen so far, so it cannot directly answer “how many are greater than X .” This calls for a different right-to-left technique – a **Fenwick tree (Binary Indexed Tree) over compressed ranks** – that still processes the array in the same right-to-left order as the rest of this section, but tracks counts rather than a skyline.

Understanding the problem carefully

Scanning right to left, by the time we reach index i , we want to know how many of the values already seen (i.e. to i ’s right) exceed $arr[i]$. This is a classic “count of elements greater than X among those inserted so far” query, repeated n times with the collection growing by one element after each query – exactly what a Fenwick tree, indexed by the *rank* of each value (after coordinate compression, since values may be arbitrary integers), is built for: it supports both “insert a value” and “count how many inserted values are $\leq$ some threshold” in $O(\log n)$ each.

Fenwick Tree over Compressed Ranks, Scanned Right to Left

Compress all values in arr to ranks $1, \dots, n$ (sorted order). Scan arr from right to left. For each $arr[i]$: query the Fenwick tree for the count of already-inserted values with rank *greater than* $arr[i]$ ’s rank (equivalently, n minus the count of values with rank $\leq arr[i]$ ’s rank) – this is exactly the number of elements to i ’s right that exceed $arr[i]$. Record this count for index i . Then insert $arr[i]$ ’s rank into the Fenwick tree, so it is correctly counted for any index further to the left.

Why scanning right to left and inserting afterward is correct

At the moment we query for index i , the Fenwick tree contains exactly the ranks of elements at indices $i + 1, \dots, n - 1$ – because we always insert $arr[i]$ *after* querying for it, and we process indices in strictly decreasing order, so every index to i ’s right has already been inserted, and

no index at or to i 's left has been inserted yet. This invariant is what makes the Fenwick tree's count query, at the moment we use it for index i , exactly answer "how many elements to the right of i exceed $arr[i]$," with no extra bookkeeping needed.

Algorithm

1. Compute ranks of all values in arr via sorting (coordinate compression).
2. Initialize an empty Fenwick tree of size n .
3. For i from $n - 1$ down to 0:
 - (a) $countGreater[i] \leftarrow n - \text{QUERY}(\text{rank}[i])$ (count of inserted values with rank $\leq \text{rank}[i]$, subtracted from the total inserted so far).
 - (b) $\text{UPDATE}(\text{rank}[i], +1)$ (insert this rank).
4. Answer each query index by direct lookup into $countGreater$.

Dry run

Dry Run. Take $arr = [3, 4, 2, 7, 5]$, query index $i = 2$ (value 2).

Ranks (1-indexed, smallest = 1): $3 \rightarrow 2, 4 \rightarrow 3, 2 \rightarrow 1, 7 \rightarrow 5, 5 \rightarrow 4$.

Scanning right to left and inserting:

- $i = 4$ (value 5, rank 4): tree empty, query returns 0 inserted so far, so $countGreater[4] = 0 - 0 = 0$. Insert rank 4.
- $i = 3$ (value 7, rank 5): inserted so far = 1 (rank 4), count with rank ≤ 5 is 1. $countGreater[3] = 1 - 1 = 0$. Insert rank 5.
- $i = 2$ (value 2, rank 1): inserted so far = 2 (ranks 4, 5), count with rank ≤ 1 is 0. $countGreater[2] = 2 - 0 = 2$. Insert rank 1.

$countGreater[2] = 2$: indeed, to the right of index 2 (value 2), the elements are 7 and 5, both greater – 2 elements, matching.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class FenwickTree {
5     vector<int> tree;
6     int n;
7 public:
8     FenwickTree(int n) : n(n), tree(n + 1, 0) {}
9
10    void update(int i, int delta) {
11        for (; i <= n; i += i & (-i)) tree[i] += delta;
12    }
13
14    int query(int i) { // count of inserted values with rank <= i
15        int sum = 0;
16        for (; i > 0; i -= i & (-i)) sum += tree[i];
17        return sum;
18    }

```

```

19 };
20
21 vector<int> countGreaterToRight(vector<int>& arr) {
22     int n = arr.size();
23     vector<int> sorted = arr;
24     sort(sorted.begin(), sorted.end());
25
26     auto rankOf = [&](int val) {
27         return lower_bound(sorted.begin(), sorted.end(), val) - sorted.
            begin() + 1;
28     };
29
30     FenwickTree ft(n);
31     vector<int> countGreater(n);
32     int insertedSoFar = 0;
33
34     for (int i = n - 1; i >= 0; i--) {
35         int r = rankOf(arr[i]);
36         countGreater[i] = insertedSoFar - ft.query(r);
37         ft.update(r, 1);
38         insertedSoFar++;
39     }
40     return countGreater;
41 }
42
43 int main() {
44     vector<int> arr = {3, 4, 2, 7, 5};
45     auto result = countGreaterToRight(arr);
46     for (int x : result) cout << x << " ";
47     cout << endl;
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. Coordinate compression costs $O(n \log n)$. The main scan performs n Fenwick tree updates and queries, each $O(\log n)$, giving an overall time complexity of

$$O(n \log n).$$

Space Complexity. $O(n)$ for the Fenwick tree and rank arrays.

Key Takeaway

“Count how many” is a structurally different question from “find the nearest,” even when both scan in the same direction – a monotonic stack answers the latter in $O(n)$ total but cannot answer the former at all, since it discards the very elements a counting query would need. Whenever a problem asks for a *count* of qualifying elements rather than the nearest one, reach for a Fenwick tree, segment tree, or balanced order-statistics structure instead of a plain stack.

1.19 Trapping Rain Water

Problem Statement. We are given n non-negative integers *height* representing an elevation map, where *height*[*i*] is the height of a bar of width 1 at position *i*. Compute how much water

is trapped after raining, assuming water can only be held between bars and runs off at either open end.

Pattern Recognition

“Water trapped between bars of varying heights” is a problem about **valleys bounded by two walls**: water above any single position is limited by the *shorter* of the nearest wall to its left and the nearest wall to its right. Whenever a problem’s answer at each position depends on “the nearest taller boundary on each side,” a monotonic stack – which is precisely a stack of candidate boundaries, each taller than the one below it – is a natural fit, processing the array once and resolving each “valley” as soon as a wall tall enough to close it off arrives.

Understanding the problem carefully

Maintain a stack of indices whose heights are non-increasing from bottom to top (each new index pushed is no taller than the one below it, otherwise it would immediately start resolving valleys). When we encounter a bar taller than the index on top of the stack, that means we have just found a **right wall** for a valley whose floor is the popped index, and whose **left wall** is whatever index remains below it on the stack. The water trapped above that floor, between the two walls, is determined by the *shorter* of the two walls (water cannot rise above the shorter wall without spilling over it) minus the floor’s own height, multiplied by the horizontal distance between the two walls.

Resolve One Valley Floor per Pop

Scan left to right. While the current bar is taller than the stack’s top: pop that top index as the *valley floor*. If the stack is now empty, there is no left wall, so this floor traps no water (stop popping for this bar). Otherwise, the new stack top is the *left wall*, and the current index is the *right wall*. The trapped water above the floor is

$$(\min(\text{height}[\text{leftWall}], \text{height}[\text{rightWall}]) - \text{height}[\text{floor}]) \times (\text{rightWall} - \text{leftWall} - 1).$$

Add this to the running total, then continue popping (the same right wall might close off multiple successively higher floors). Finally, push the current index.

Why resolving level by level, floor by floor, correctly sums all trapped water

Every unit of trapped water sits directly above some specific bar (the “floor”) and below the line connecting the nearest sufficiently tall walls on either side. By processing each floor exactly once – the moment a right wall taller than it is found, with whatever remains on the stack serving as the left wall – we account for the entire water column above that floor in one $O(1)$ computation. Because indices are only ever pushed once and popped at most once, every floor’s contribution is computed exactly once, and the total accumulated is exactly the total trapped water, with no double-counting and no gaps.

Algorithm

1. Initialize an empty stack and $\text{totalWater} \leftarrow 0$.
2. For i from 0 to $n - 1$:
 - (a) While the stack is not empty and $\text{height}[i] > \text{height}[\text{stack's top}]$:
 - i. Pop the top as *floor*.
 - ii. If the stack is now empty: break (no left wall).

- iii. $leftWall \leftarrow$ the new top; $width \leftarrow i - leftWall - 1$; $boundedHeight \leftarrow \min(height[leftWall], height[i]) - height[floor]$.
- iv. $totalWater \leftarrow totalWater + width \times boundedHeight$.

(b) Push i .

3. Return $totalWater$.

Dry run

Dry Run. Take $height = [3, 0, 2, 0, 4]$.

- $i = 0$ ($h = 3$): stack empty. Push 0. Stack = [0].
- $i = 1$ ($h = 0$): $0 \leq height[0] = 3$, no pop. Push 1. Stack = [0, 1].
- $i = 2$ ($h = 2$): $2 > height[1] = 0$, pop floor = 1. Stack = [0], not empty: leftWall = 0. Width = $2 - 0 - 1 = 1$. boundedHeight = $\min(3, 2) - 0 = 2$. Water += $1 \times 2 = 2$. $totalWater = 2$. Now $2 \leq height[0] = 3$, stop popping. Push 2. Stack = [0, 2].
- $i = 3$ ($h = 0$): $0 \leq height[2] = 2$, no pop. Push 3. Stack = [0, 2, 3].
- $i = 4$ ($h = 4$): $4 > height[3] = 0$, pop floor = 3. Left wall = 2. Width = $4 - 2 - 1 = 1$. boundedHeight = $\min(2, 4) - 0 = 2$. Water += $1 \times 2 = 2$. Total = 4. Continue: $4 > height[2] = 2$, pop floor = 2. Left wall = 0. Width = $4 - 0 - 1 = 3$. boundedHeight = $\min(3, 4) - 2 = 1$. Water += $3 \times 1 = 3$. Total = 7. Now $4 > height[0] = 3$, pop floor = 0. Stack empty – break (no left wall). Push 4.

Total trapped water: 7.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int trapRainWater(vector<int>& height) {
5      int n = height.size();
6      stack<int> st;
7      long long totalWater = 0;
8
9      for (int i = 0; i < n; i++) {
10         while (!st.empty() && height[i] > height[st.top()]) {
11             int floorIdx = st.top();
12             st.pop();
13             if (st.empty()) break;
14
15             int leftWall = st.top();
16             int width = i - leftWall - 1;
17             int boundedHeight = min(height[leftWall], height[i]) -
18                                     height[floorIdx];
19             totalWater += (long long)width * boundedHeight;
20         }
21         st.push(i);
22     }
23     return totalWater;
24 }
```

```

25 int main() {
26     vector<int> height = {3, 0, 2, 0, 4};
27     cout << "Trapped water: " << trapRainWater(height) << endl;
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. Each index is pushed once and popped at most once, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack in the worst case. (A two-pointer technique solves this same problem in $O(1)$ extra space, by tracking running left-max and right-max boundaries directly – worth knowing as a further optimization, though the stack-based derivation here is the one that generalizes most directly to the histogram-area problems in Sections 1.24–1.25.)

Key Takeaway

“Water/area bounded by the nearest sufficiently tall walls on both sides” is solved by a monotonic stack that resolves one bounded region per pop, the instant a wall tall enough to close it off arrives. This same skeleton – pop a floor, look at what remains as the left boundary, the trigger as the right boundary, compute a bounded quantity, accumulate – reappears in Largest Rectangle in Histogram (Section 1.24).

1.20 Sum of Subarray Minimums

Problem Statement. Given an array arr of n integers, consider every one of its $\binom{n+1}{2}$ contiguous subarrays. Compute the sum, over all of them, of each subarray’s minimum element (the result can be large, so it is typically returned modulo $10^9 + 7$).

Pattern Recognition

“Sum, over all subarrays, of each subarray’s minimum (or maximum)” is a strong signal to flip the entire computation around: instead of asking, for each subarray, “what is its minimum,” ask, for each **element**, “in how many subarrays is *this* element the minimum.” This contribution-counting reformulation, combined with a monotonic stack to compute exactly that count for every element in $O(n)$ total, avoids ever enumerating the $O(n^2)$ subarrays directly.

Understanding the problem carefully

For element $arr[i]$ to be the minimum of some subarray, that subarray must be entirely contained within the maximal range around i over which $arr[i]$ remains the smallest value – bounded on the left by the nearest strictly smaller element, and on the right by the nearest smaller-or-equal element (using a strict boundary on one side and a non-strict boundary on the other is essential to avoid double-counting subarrays when duplicate values are present, by consistently attributing each subarray’s minimum to a single, well-defined occurrence among any tied values).

Count Subarrays Where $arr[i]$ Is the Minimum

For each i , let $left[i]$ be the number of valid choices for a subarray's left endpoint such that every element from that endpoint up to i is $\geq arr[i]$ – this equals $i - PLE(i)$, where $PLE(i)$ is the index of the nearest element to the left that is *strictly less* than $arr[i]$ (or -1 if none). Let $right[i]$ be the number of valid choices for the right endpoint such that every element from i to that endpoint is $\geq arr[i]$ – this equals $NLE(i) - i$, where $NLE(i)$ is the nearest element to the right that is *less than or equal to* $arr[i]$ (or n if none). Then $arr[i]$ is the minimum of exactly $left[i] \times right[i]$ subarrays (any combination of a valid left endpoint and a valid right endpoint).

Why the strict/non-strict boundary asymmetry avoids double-counting

If both boundaries used the same (either both strict, or both non-strict) comparison, a subarray containing two equal minimum values would be counted once for *each* occurrence of that value, inflating the sum. Using strictly-less on the left and less-or-equal on the right ensures that, among several equal candidate minimum positions within a subarray, only the **leftmost** one “claims” that subarray as contributing to its count: the leftmost equal occurrence's right boundary search will stop at the very next equal (or smaller) value, while any later equal occurrence's left boundary search will pass straight over the earlier equal value (since it is not *strictly less*) and only stop at a genuinely smaller one further left – so the two occurrences never both count the same subarray.

Algorithm

1. Compute $left[i]$ for every i via a monotonic increasing stack scanned left to right: pop while the stack's top value $\geq arr[i]$; $left[i] \leftarrow i - (\text{stack empty?} - 1 : \text{top index})$; push i .
2. Compute $right[i]$ for every i via a monotonic stack scanned right to left: pop while the stack's top value $> arr[i]$; $right[i] \leftarrow (\text{stack empty?} n : \text{top index}) - i$; push i .
3. Sum $arr[i] \times left[i] \times right[i]$ over all i , modulo $10^9 + 7$.

Dry run

Dry Run. Take $arr = [3, 1, 2, 4]$.

Left (pop while top $\geq arr[i]$): $left = [1, 2, 1, 1]$ (computed via the standard PLE scan, e.g. at $i = 2$, value 2: the stack's top is index 1, value $1 < 2$, so no pop; $left[2] = 2 - 1 = 1$).

Right (pop while top $> arr[i]$): $right = [1, 3, 2, 1]$ (e.g. at $i = 1$, value 1: scanning right to left, both index 2 (value 2) and index 3... after appropriate pops the stack empties, giving $right[1] = 4 - 1 = 3$).

Contributions: $3 \times 1 \times 1 = 3$; $1 \times 2 \times 3 = 6$; $2 \times 1 \times 2 = 4$; $4 \times 1 \times 1 = 4$. Total = $3 + 6 + 4 + 4 = 17$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 int sumSubarrayMins(vector<int>& arr) {
6     int n = arr.size();
7     vector<int> left(n), right(n);
8     stack<int> st;
```

```

9
10     for (int i = 0; i < n; i++) {
11         while (!st.empty() && arr[st.top()] >= arr[i]) st.pop();
12         left[i] = i - (st.empty() ? -1 : st.top());
13         st.push(i);
14     }
15
16     while (!st.empty()) st.pop();
17
18     for (int i = n - 1; i >= 0; i--) {
19         while (!st.empty() && arr[st.top()] > arr[i]) st.pop();
20         right[i] = (st.empty() ? n : st.top()) - i;
21         st.push(i);
22     }
23
24     long long total = 0;
25     for (int i = 0; i < n; i++) {
26         total = (total + (long long)arr[i] * left[i] % MOD * right[i])
27             % MOD;
28     }
29     return total;
30 }
31
32 int main() {
33     vector<int> arr = {3, 1, 2, 4};
34     cout << "Sum of subarray minimums: " << sumSubarrayMins(arr) <<
35         endl;
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. Two monotonic-stack passes, each $O(n)$, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stacks and the *left/right* arrays.

Key Takeaway

“Sum/count over all subarrays of [some per-subarray extremum]” is solved not by enumerating subarrays, but by computing, for each element, *how many* subarrays it is the extremum of – a count derived from the nearest strictly-more-extreme boundaries on each side, found with two monotonic-stack passes. This contribution-counting reformulation is one of the most powerful general techniques to recognize in this chapter.

1.21 Sum of Subarray Ranges

Problem Statement. Given an array *arr*, the **range** of a subarray is defined as its maximum element minus its minimum element. Compute the sum of ranges over all contiguous subarrays.

Pattern Recognition

A subarray's range is, by definition, $\max - \min$. Summing ranges over all subarrays is therefore exactly

$$\left(\sum_{\text{subarrays}} \max \right) - \left(\sum_{\text{subarrays}} \min \right),$$

and the second term is precisely Section 1.20's problem. The first term is its exact mirror image – “sum of subarray maximums” – solved by the same contribution-counting technique with every comparison flipped. No new derivation is needed; this problem is a direct combination of two near-identical applications of the same idea.

Understanding the problem carefully

Linearity of summation lets us split the sum of $(\max - \min)$ over all subarrays into a sum of $\max$ over all subarrays, minus a sum of $\min$ over all subarrays, computed completely independently. Each piece is solved by the contribution-counting technique from Section 1.20: for the minimum part, exactly as before; for the maximum part, the same left/right boundary counting, but searching for the nearest strictly-*greater* element (instead of strictly-less) on one side and greater-or-equal on the other.

Reuse Twice, with Comparisons Flipped for the Maximum Half

Compute *sumOfMins* exactly as in Algorithm 1.20. Compute *sumOfMaxs* with the same algorithm, but with stack comparisons flipped: the left pass pops while the stack's top value $\leq \text{arr}[i]$ (finding the nearest strictly *greater* element to the left), and the right pass pops while the stack's top value $< \text{arr}[i]$ (finding the nearest greater-or-equal element to the right). Return *sumOfMaxs* – *sumOfMins*.

Why flipping every comparison correctly computes the maximum-side sum

This is a direct restatement of the next-greater/next-smaller duality established in Section 1.17: “how many subarrays is $\text{arr}[i]$ the maximum of” is structurally identical to “how many subarrays is $\text{arr}[i]$ the minimum of,” with every $<$ replaced by $>$ and vice versa. The strict/non-strict asymmetry between the two boundary searches is still required, for the same reason as Section 1.20: to avoid double-counting subarrays containing duplicate maximum values.

Algorithm

1. Compute *sumOfMins* using Algorithm 1.20 unchanged.
2. Compute *sumOfMaxs* using the same algorithm structure, with left-pass pops on $\leq$ and right-pass pops on $<$ (instead of $\geq$ and $>$ respectively).
3. Return *sumOfMaxs* – *sumOfMins*.

Dry run

Dry Run. Take $\text{arr} = [1, 2, 3]$.

sumOfMins: subarrays and their minimums: $[1]=1, [2]=2, [3]=3, [1,2]=1, [2,3]=2, [1,2,3]=1$.
Sum = $1 + 2 + 3 + 1 + 2 + 1 = 10$.

sumOfMaxs: maximums: $1, 2, 3, 2, 3, 3$. Sum = $1 + 2 + 3 + 2 + 3 + 3 = 14$.

Sum of ranges = $14 - 10 = 4$. Checking directly: ranges are $0, 0, 0, 1, 1, 2$, summing to 4. Matches.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long sumOfMins(vector<int>& arr) {
5      int n = arr.size();
6      vector<int> left(n), right(n);
7      stack<int> st;
8
9      for (int i = 0; i < n; i++) {
10         while (!st.empty() && arr[st.top()] >= arr[i]) st.pop();
11         left[i] = i - (st.empty() ? -1 : st.top());
12         st.push(i);
13     }
14     while (!st.empty()) st.pop();
15     for (int i = n - 1; i >= 0; i--) {
16         while (!st.empty() && arr[st.top()] > arr[i]) st.pop();
17         right[i] = (st.empty() ? n : st.top()) - i;
18         st.push(i);
19     }
20
21     long long total = 0;
22     for (int i = 0; i < n; i++) total += (long long)arr[i] * left[i] *
23         right[i];
24     return total;
25 }
26
27 long long sumOfMaxs(vector<int>& arr) {
28     int n = arr.size();
29     vector<int> left(n), right(n);
30     stack<int> st;
31
32     for (int i = 0; i < n; i++) {
33         while (!st.empty() && arr[st.top()] <= arr[i]) st.pop();
34         left[i] = i - (st.empty() ? -1 : st.top());
35         st.push(i);
36     }
37     while (!st.empty()) st.pop();
38     for (int i = n - 1; i >= 0; i--) {
39         while (!st.empty() && arr[st.top()] < arr[i]) st.pop();
40         right[i] = (st.empty() ? n : st.top()) - i;
41         st.push(i);
42     }
43
44     long long total = 0;
45     for (int i = 0; i < n; i++) total += (long long)arr[i] * left[i] *
46         right[i];
47     return total;
48 }
49
50 long long sumSubarrayRanges(vector<int>& arr) {
51     return sumOfMaxs(arr) - sumOfMins(arr);
52 }
53
54 int main() {
55     vector<int> arr = {1, 2, 3};

```

```

54     cout << "Sum of subarray ranges: " << sumSubarrayRanges(arr) <<
        endl;
55     return 0;
56 }

```

Complexity Analysis

Time Complexity. Four monotonic-stack passes total (two for minimums, two for maximums), each $O(n)$, giving

$$O(n).$$

Space Complexity. $O(n)$.

Key Takeaway

When a quantity decomposes linearly (here, $\text{range} = \text{max} - \text{min}$), compute each piece with whatever technique already solves it, and combine at the end – there is no need to find a single unified algorithm that handles max and min simultaneously when summing them separately and subtracting is just as correct and far simpler to reason about.

1.22 Asteroid Collision

Problem Statement. We are given an array *asteroids*, where each element’s absolute value is an asteroid’s size and its sign is its direction of travel (positive moves right, negative moves left; all move at the same speed). When two asteroids meet, the smaller one explodes; if both are the same size, both explode. Asteroids moving in the same direction never meet. Return the state of the asteroids after all collisions have finished.

Pattern Recognition

“Process a sequence left to right, where a new element can repeatedly ‘defeat’ and remove previously-kept elements (or be defeated by them) based on a comparison” is, structurally, the same skeleton as next greater element (Section 1.15): a stack of “currently surviving” items, where each new arrival potentially knocks out some suffix of what’s already on the stack before either joining it or being knocked out itself.

Understanding the problem carefully

A collision can only happen between a **right-moving** asteroid that was placed earlier and a **left-moving** asteroid that arrives later (two right-movers never catch up to each other since they move at the same speed; a left-mover followed by a right-mover are moving apart and never meet). So, scanning left to right and keeping a stack of asteroids that have survived so far: whenever a new *left-moving* asteroid arrives and the stack’s top is a surviving *right-moving* asteroid, they are on a collision course, and we must resolve who survives – possibly triggering a chain of further collisions if the new asteroid is large enough to destroy multiple right-movers in a row.

Stack of Surviving Asteroids, with Chain Collisions

Maintain a stack. For a right-moving asteroid (positive), simply push it – no immediate collision is possible yet. For a left-moving asteroid (negative): while the stack’s top is positive (a right-mover) and smaller in size than the incoming asteroid, pop it (it loses). If the stack’s top is positive and exactly equal in size, pop it and the incoming asteroid is also destroyed (mutual annihilation) – stop processing this asteroid. If the stack’s top is positive

and larger, the incoming asteroid is destroyed – stop processing this asteroid, do not push it. If the stack becomes empty or its top is negative (no more right-movers to collide with), and the incoming asteroid was never destroyed, push it.

Why processing collisions as a chain (not just one comparison) is correct

A single incoming left-moving asteroid might be large enough to destroy *several* consecutive right-moving asteroids already on the stack before either finally being destroyed itself or surviving to be pushed. Each such destruction is resolved by exactly the comparison rule above, applied repeatedly – this is precisely the same “pop everything the new arrival invalidates” loop structure as a monotonic stack, except the “invalidation” rule here is a size comparison between asteroids of opposite direction rather than a simple value comparison, and the loop must also account for the three-way outcome (incoming destroyed, top destroyed, or both destroyed) rather than a single binary pop/no-pop decision.

Algorithm

1. Initialize an empty stack.
2. For each asteroid a in *asteroids*:
 - (a) $alive \leftarrow \text{true}$.
 - (b) While $alive$, $a < 0$, the stack is not empty, and the stack's top is > 0 :
 - If $\text{top} < -a$: pop (the top asteroid is destroyed); continue the loop.
 - Else if $\text{top} = -a$: pop; $alive \leftarrow \text{false}$ (mutual destruction).
 - Else ($\text{top} > -a$): $alive \leftarrow \text{false}$ (the incoming asteroid is destroyed).
 - (c) If $alive$: push a .
3. Return the stack's contents, bottom to top.

Dry run

Dry Run. Take $asteroids = [5, 10, -5]$.

- $a = 5$: positive, push. Stack = $[5]$.
- $a = 10$: positive, push. Stack = $[5, 10]$.
- $a = -5$: negative. Top is $10 > 0$. Is $10 < 5$? No. Is $10 = 5$? No. So top (10) > 5 : the incoming asteroid (-5) is destroyed. $alive = \text{false}$. Do not push.

Final stack: $[5, 10]$.

Now take $asteroids = [8, -8]$: $a = 8$ pushed. $a = -8$: top is 8 , and $8 = 8$: pop, mutual destruction, $alive = \text{false}$. Final stack: $[]$ (both annihilated).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> asteroidCollision(vector<int>& asteroids) {
5     vector<int> st; // used as a stack via back()/push_back()/pop_back
6     ()

```

```

6
7     for (int a : asteroids) {
8         bool alive = true;
9         while (alive && a < 0 && !st.empty() && st.back() > 0) {
10             if (st.back() < -a) {
11                 st.pop_back(); // top destroyed, continue checking
12             } else if (st.back() == -a) {
13                 st.pop_back();
14                 alive = false; // mutual destruction
15             } else {
16                 alive = false; // incoming asteroid destroyed
17             }
18         }
19         if (alive) st.push_back(a);
20     }
21     return st;
22 }
23
24 int main() {
25     vector<int> asteroids = {5, 10, -5};
26     for (int x : asteroidCollision(asteroids)) cout << x << " ";
27     cout << endl;
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. Each asteroid is pushed at most once and popped at most once across the entire scan, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack in the worst case (no collisions occur).

Key Takeaway

Monotonic-stack-style “pop everything the new arrival invalidates” logic generalizes beyond simple value comparisons: here the invalidation rule is a three-way size comparison between opposite-direction asteroids, but the overall shape – a while-loop popping a suffix of the stack, then conditionally pushing the new arrival – is unchanged from Section 1.15.

1.23 Remove K Digits

Problem Statement. Given a non-negative integer represented as a string *num*, and an integer *k*, remove exactly *k* digits from *num* so that the remaining digits, kept in their original relative order, form the **smallest possible** number. The result must not have leading zeros (unless it is exactly “0”).

Pattern Recognition

“Remove *k* elements (keeping relative order) to make the result as small/large as possible” is a greedy-with-a-stack pattern: a number is smaller when an earlier (more significant) digit is smaller, so whenever we see a digit smaller than what we have just placed, removing that just-placed (now regretted) larger digit is always an improvement – a monotonic increasing stack of kept digits, with removals funded by the budget *k*, captures exactly this.

Understanding the problem carefully

Among any two digit strings of the same length, the one with the smaller digit at the first position where they differ is the smaller number – so, greedily, whenever we have the opportunity to remove a digit and thereby make some earlier-kept digit smaller (by exposing a smaller digit to take its place at that position), we should take it, as long as we still have removals left in our budget k .

Monotonic Increasing Stack of Kept Digits, Funded by Budget k

Scan num left to right, maintaining a stack of digits kept so far. For each new digit d : while $k > 0$ and the stack is non-empty and its top digit exceeds d , pop the top (spend one unit of k) – this digit was a mistake to keep, now that a smaller one is available to take its place. Then push d . After the scan, if k still has unused budget, remove that many digits from the *end* of the stack (the largest remaining suffix, since no smaller digit ever showed up to justify removing them earlier). Finally, strip any leading zeros, and return "0" if the result is empty.

Why popping a larger top whenever a smaller digit arrives is optimal

Removing a digit only ever helps if it lets an earlier, larger digit be replaced (in significance/-position) by a smaller later digit – and the earliest opportunity to do this should always be taken, since a smaller digit at a *more significant* (more leftward) position always beats any improvement achievable further right. This is exactly the same exchange-argument logic as every monotonic-stack pop rule in this chapter: keeping a larger digit when a smaller one is available and removals remain in budget can never be better than removing it, since the resulting number would be lexicographically (and numerically, for same-length strings) larger or equal in every case.

Algorithm

1. Initialize an empty stack of digits.
2. For each digit d in num :
 - (a) While $k > 0$ and the stack is non-empty and its top $> d$: pop; $k \leftarrow k - 1$.
 - (b) Push d .
3. While $k > 0$: pop from the end of the stack; $k \leftarrow k - 1$.
4. Strip leading zeros from the stack's contents (read bottom to top). If the result is empty, return "0".

Dry run

Dry Run. Take $num = "1432219"$, $k = 3$.

- 1: stack empty, push. Stack = [1].
- 4: top 1 < 4, no pop. Push. Stack = [1, 4].
- 3: top 4 > 3 and $k = 3 > 0$: pop 4, $k = 2$. New top 1 < 3, stop. Push 3. Stack = [1, 3].
- 2: top 3 > 2 and $k = 2 > 0$: pop 3, $k = 1$. New top 1 < 2, stop. Push 2. Stack = [1, 2].
- 2: top 2 $\not>$ 2, no pop. Push. Stack = [1, 2, 2].

- 1: $\text{top } 2 > 1$ and $k = 1 > 0$: pop 2, $k = 0$. New top $2 > 1$ but $k = 0$ now, stop. Push 1. Stack = [1, 2, 1].
- 9: $k = 0$, no pop. Push. Stack = [1, 2, 1, 9].

$k = 0$, no trailing removal needed. Result (no leading zeros to strip): "1219".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string removeKdigits(string num, int k) {
5     string st; // used as a stack of characters
6
7     for (char d : num) {
8         while (k > 0 && !st.empty() && st.back() > d) {
9             st.pop_back();
10            k--;
11        }
12        st.push_back(d);
13    }
14
15    while (k > 0 && !st.empty()) {
16        st.pop_back();
17        k--;
18    }
19
20    int i = 0;
21    while (i < (int)st.size() - 1 && st[i] == '0') i++;
22    st = st.substr(i);
23
24    return st.empty() ? "0" : st;
25 }
26
27 int main() {
28     cout << removeKdigits("1432219", 3) << endl;
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. Each digit is pushed once and popped at most once, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack.

Key Takeaway

“Remove exactly k elements to minimize/maximize the resulting sequence, preserving relative order” is solved by a monotonic stack where the popping budget is explicitly k – once k reaches zero, the stack stops discarding even objectively “bad” elements, and any leftover budget at the end is spent removing from the tail, since nothing better ever arrived to justify earlier removals there.

1.24 Largest Rectangle in Histogram

Problem Statement. Given an array *heights* representing the heights of adjacent bars of width 1 in a histogram, find the area of the **largest rectangle** that can be formed using contiguous bars (the rectangle's height is limited by the shortest bar it spans).

Pattern Recognition

This is the contribution-counting idea from Section 1.20, repurposed to **maximize** rather than sum: for every bar i , the largest rectangle that uses $heights[i]$ as its limiting height spans exactly the maximal contiguous range around i over which $heights[i]$ is the minimum – found by the exact same nearest strictly-smaller-element boundaries on each side.

Understanding the problem carefully

For a rectangle of height $heights[i]$ to be valid, every bar it spans must be at least $heights[i]$ tall (otherwise the rectangle would poke above some shorter bar within its width). So the widest rectangle using $heights[i]$ as its height extends left until hitting a strictly shorter bar (or the histogram's edge), and right until hitting a strictly shorter bar (or the edge) – exactly the $left[i]$ and $right[i]$ boundaries from Section 1.20, just used to compute one candidate area per bar and take the maximum, rather than summing contributions.

One Pop, One Area, via a Single Monotonic Pass

Maintain a stack of indices with strictly increasing heights. Scan left to right (appending a sentinel height of 0 at the very end to force every remaining bar to be resolved). When the current height is less than the stack's top height, pop that top index j : the rectangle with height $heights[j]$ has width $i - (\text{new top index}) - 1$ (or simply i if the stack becomes empty, meaning j 's rectangle could extend all the way to the histogram's left edge). Compute this area and update the running maximum. Continue popping until the current height no longer triggers a pop, then push the current index.

Why a single left-to-right pass (with a sentinel) suffices

This combines the boundary-finding idea from Section 1.20 with the floor-resolution idea from Trapping Rain Water (Section 1.19): each bar's rectangle is fully resolved – both its left and right boundary determined – in a single pop, because by the time bar j is popped, the current index i is exactly its right boundary (the nearest strictly shorter bar to the right), and the stack's new top after the pop is exactly its left boundary (the nearest still-standing, and therefore strictly shorter or absent, bar to the left). The sentinel height 0 at the end guarantees every bar still on the stack when the scan finishes is popped and resolved, rather than left unresolved.

Algorithm

1. Append a sentinel value 0 to the end of *heights* (conceptually); initialize an empty stack and $maxArea \leftarrow 0$.
2. For i from 0 to n (inclusive of the sentinel position):
 - (a) While the stack is not empty and $heights[i] < heights[\text{stack's top}]$:
 - Pop the top as j .
 - $width \leftarrow (\text{stack empty} ? i : i - \text{new top} - 1)$.
 - $maxArea \leftarrow \max(maxArea, heights[j] \times width)$.

(b) Push i .

3. Return $maxArea$.

Dry run

Dry Run. Take $heights = [2, 1, 5, 6, 2, 3]$ (sentinel 0 appended at index 6).

- $i = 0$ ($h = 2$): stack empty, push. Stack = [0].
- $i = 1$ ($h = 1$): $1 < 2$, pop 0. Stack empty: width = 1. Area = $2 \times 1 = 2$. $maxArea = 2$. Push 1. Stack = [1].
- $i = 2$ ($h = 5$): $5 \not< 1$, no pop. Push. Stack = [1, 2].
- $i = 3$ ($h = 6$): $6 \not< 5$, no pop. Push. Stack = [1, 2, 3].
- $i = 4$ ($h = 2$): $2 < 6$, pop 3: width = $4 - 2 - 1 = 1$. Area = $6 \times 1 = 6$. $maxArea = 6$. $2 < 5$, pop 2: width = $4 - 1 - 1 = 2$. Area = $5 \times 2 = 10$. $maxArea = 10$. $2 \not< 1$, stop. Push 4. Stack = [1, 4].
- $i = 5$ ($h = 3$): $3 \not< 2$, no pop. Push. Stack = [1, 4, 5].
- $i = 6$ ($h = 0$, sentinel): $0 < 3$, pop 5: width = $6 - 4 - 1 = 1$. Area = 3. $0 < 2$, pop 4: width = $6 - 1 - 1 = 4$. Area = 8. $0 < 1$, pop 1: stack empty: width = 6. Area = 6. None exceed 10.

The largest rectangle has area 10 (bars of height 5 and 6 at indices 2, 3, using height 5 across width 2).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int largestRectangleArea(vector<int>& heights) {
5      heights.push_back(0); // sentinel
6      int n = heights.size();
7      stack<int> st;
8      long long maxArea = 0;
9
10     for (int i = 0; i < n; i++) {
11         while (!st.empty() && heights[i] < heights[st.top()]) {
12             int j = st.top();
13             st.pop();
14             int width = st.empty() ? i : i - st.top() - 1;
15             maxArea = max(maxArea, (long long)heights[j] * width);
16         }
17         st.push(i);
18     }
19     heights.pop_back(); // restore original array
20     return maxArea;
21 }
22
23 int main() {
24     vector<int> heights = {2, 1, 5, 6, 2, 3};
25     cout << "Largest rectangle area: "
```

```

26         << largestRectangleArea(heights) << endl;
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. Each index is pushed once and popped at most once, giving

$$O(n).$$

Space Complexity. $O(n)$ for the stack.

Key Takeaway

“Largest rectangle/area limited by the shortest bar within some span” is solved by the same boundary-finding idea as Section 1.20, condensed into a single left-to-right pass with a sentinel, since we only need each bar’s *maximum* possible area, not a sum over every possible sub-span. This single-pass technique is reused directly in Maximal Rectangle (Section 1.25) to extend from a 1D histogram to a full 2D binary matrix.

1.25 Maximal Rectangle

Problem Statement. Given an $n \times m$ binary matrix, find the area of the largest rectangle that contains only 1s.

Pattern Recognition

“Largest all-1s rectangle in a binary matrix” is Largest Rectangle in Histogram (Section 1.24) applied **once per row**, where each row defines a histogram of “how tall a column of consecutive 1s ends here.” The keyword to notice: a 2D “maximal rectangle of some property” problem often reduces to a 1D technique applied row by row, with state carried forward between rows.

Understanding the problem carefully

For each row r , define $height[j]$ as the number of consecutive 1s ending at row r in column j , looking upward (so $height[j] = 0$ if $matrix[r][j] = 0$, and $height[j] = height_{\text{previous row}}[j] + 1$ if $matrix[r][j] = 1$). This $height$ array is exactly a histogram: a rectangle of all 1s that has its *bottom edge* on row r corresponds exactly to a rectangle in this histogram, since the histogram’s height at column j tells us exactly how far upward a solid column of 1s extends from row r .

Run Largest-Rectangle-in-Histogram Once per Row

Maintain a running $height$ array across rows, updating it as described above after moving to each new row. After updating $height$ for row r , run Algorithm 1.24’s histogram algorithm on it, and record the resulting maximum area as a candidate for the overall answer. After processing every row, the largest area seen across all rows is the answer.

Why checking every row’s histogram captures every possible rectangle

Every all-1s rectangle in the matrix has *some* specific bottom row r . When we process row r , the running $height$ array at that point correctly reflects, for every column, how far a solid run of 1s extends upward from row r – so the histogram-area algorithm run on that exact $height$

array is guaranteed to find the largest rectangle with bottom edge on row r specifically. Since every possible all-1s rectangle has its bottom edge on *some* row, running this check for every row and taking the overall maximum is guaranteed not to miss any candidate rectangle.

Algorithm

1. Initialize $height[0..m-1] \leftarrow 0$ and $maxArea \leftarrow 0$.
2. For each row r from 0 to $n-1$:
 - (a) For each column j : if $matrix[r][j] = 1$: $height[j] \leftarrow height[j] + 1$; else: $height[j] \leftarrow 0$.
 - (b) Run the largest-rectangle-in-histogram algorithm on $height$; update $maxArea$ with the result.
3. Return $maxArea$.

Dry run

Dry Run. Take the matrix $\begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$.

- Row 0: $height = [1, 0, 1, 0, 0]$. Largest histogram rectangle: each isolated bar of height 1, area 1. $maxArea = 1$.
- Row 1: $height = [2, 0, 2, 1, 1]$. Largest rectangle: bars at indices 2, 3, 4 with heights 2, 1, 1 give area $1 \times 3 = 3$ (limited by the shortest, height 1); also indices 0 alone gives 2; index 2 alone gives 2. Best here is 3. $maxArea = 3$.
- Row 2: $height = [3, 1, 3, 2, 2]$. Largest rectangle: indices 2, 3, 4 with heights 3, 2, 2 give area $2 \times 3 = 6$. $maxArea = 6$.
- Row 3: $height = [4, 0, 0, 3, 0]$. Largest rectangle: index 0 alone gives 4; index 3 alone gives 3. Best here is 4, not exceeding 6.

The maximal rectangle has area $\boxed{6}$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int largestRectangleArea(vector<int> heights) {
5     heights.push_back(0);
6     int n = heights.size();
7     stack<int> st;
8     long long maxArea = 0;
9
10    for (int i = 0; i < n; i++) {
11        while (!st.empty() && heights[i] < heights[st.top()]) {
12            int j = st.top();
13            st.pop();
14            int width = st.empty() ? i : i - st.top() - 1;
15            maxArea = max(maxArea, (long long)heights[j] * width);
16        }
    }

```

```

17         st.push(i);
18     }
19     return maxArea;
20 }
21
22 int maximalRectangle(vector<vector<int>>& matrix) {
23     int n = matrix.size(), m = matrix[0].size();
24     vector<int> height(m, 0);
25     int maxArea = 0;
26
27     for (int r = 0; r < n; r++) {
28         for (int j = 0; j < m; j++) {
29             height[j] = (matrix[r][j] == 1) ? height[j] + 1 : 0;
30         }
31         maxArea = max(maxArea, largestRectangleArea(height));
32     }
33     return maxArea;
34 }
35
36 int main() {
37     vector<vector<int>> matrix = {
38         {1,0,1,0,0},
39         {1,0,1,1,1},
40         {1,1,1,1,1},
41         {1,0,0,1,0}
42     };
43     cout << "Maximal rectangle area: " << maximalRectangle(matrix) <<
44         endl;
45     return 0;
46 }

```

Complexity Analysis

Time Complexity. Updating the height array for each row costs $O(m)$, and running the histogram algorithm costs $O(m)$, for a total of $O(m)$ per row across n rows, giving

$$O(nm).$$

Space Complexity. $O(m)$ for the running height array and the histogram algorithm's stack.

Key Takeaway

A 2D “maximal region with some property” problem often decomposes into “run a 1D algorithm on a derived array, once per row (or column), with the derived array updated incrementally as you move between rows.” Once you have Largest Rectangle in Histogram, Maximal Rectangle requires no new stack logic at all – only the row-by-row height book-keeping is new.

1.26 Stock Span Problem

We now turn to the final group of problems in this chapter: a handful of practical implementation and design problems that each combine stacks or queues with a hash map or other auxiliary structure to solve a real-world-flavored task efficiently.

Problem Statement. We are given the price of a stock on successive days, arriving one at a time (a stream). For each new day's price, compute its **span**: the number of consecutive days, ending with and including today, for which today's price is greater than or equal to each of those days' prices.

Pattern Recognition

"For each new arrival, look backward and count a contiguous run based on a comparison" is the *previous* greater/smaller-element family from Section 1.17, but with a twist: we need a **count** (the span length), not just the nearest qualifying index. The key efficiency idea is that, instead of storing every individual day on the stack, we can store **(price, span) pairs** and merge spans together as we pop – avoiding ever re-scanning days that have already been absorbed into a previous day's span.

Understanding the problem carefully

The span of today's price is 1 (counting today itself) plus the spans of every immediately preceding day whose price is $\leq$ today's price, chained backward as far as this holds. If we stored only raw prices on the stack, computing this span naively would require popping one day at a time and could cost $O(n)$ per query in the worst case, but with no way to skip over runs that were already resolved. Storing the *span alongside* each price lets us absorb an entire previously-resolved run in a single pop, rather than one day at a time.

Stack of (Price, Span) Pairs, Merging on Pop

Maintain a stack of $(price, span)$ pairs. For each new price p : initialize $currentSpan \leftarrow 1$. While the stack is not empty and its top's price $\leq p$: pop the top, and add its $span$ to $currentSpan$ (this whole already-resolved run is absorbed in one step, not re-examined day by day). Push $(p, currentSpan)$, and report $currentSpan$ as today's span.

Why absorbing a popped pair's entire span (not just its price) is correct

If the top of the stack is a pair $(price', span')$ with $price' \leq p$, then *every* day within that pair's already-computed span also has a price $\leq price' \leq p$ (that is precisely what the pair's span certifies, by the inductive construction of the algorithm) – so all of those days extend today's span too, and we can add the whole count $span'$ in one step rather than re-deriving it. This amortizes the total work: each day is pushed exactly once and, across its entire lifetime, can only be merged away (absorbed into a later pair) once, giving linear total work despite some individual days having large spans.

Algorithm

1. Initialize an empty stack of (price, span) pairs.
2. For each new price p :
 - (a) $currentSpan \leftarrow 1$.
 - (b) While the stack is not empty and its top's price $\leq p$: pop $(price', span')$; $currentSpan \leftarrow currentSpan + span'$.
 - (c) Push $(p, currentSpan)$.
 - (d) Report $currentSpan$.

Dry run

Dry Run. Prices arrive in order: 100, 80, 60, 70, 60, 75, 85.

- 100: stack empty. span= 1. Push (100,1). Report 1.
- 80: top (100,1), price 100 > 80, no merge. span= 1. Push (80,1). Report 1.
- 60: top (80,1), 80 > 60, no merge. span= 1. Push (60,1). Report 1.
- 70: top (60,1), 60 ≤ 70: merge, span= 1 + 1 = 2, pop. New top (80,1), 80 > 70, stop. Push (70,2). Report 2.
- 60: top (70,2), 70 > 60, no merge. span= 1. Push (60,1). Report 1.
- 75: top (60,1), 60 ≤ 75: merge, span= 1 + 1 = 2, pop. Top (70,2), 70 ≤ 75: merge, span= 2 + 2 = 4, pop. Top (80,1), 80 > 75, stop. Push (75,4). Report 4.
- 85: top (75,4), 75 ≤ 85: merge, span= 1 + 4 = 5, pop. Top (80,1), 80 ≤ 85: merge, span= 5 + 1 = 6, pop. Top (100,1), 100 > 85, stop. Push (85,6). Report 6.

Spans: [1, 1, 1, 2, 1, 4, 6].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class StockSpanner {
5     stack<pair<int,int>> st; // (price, span)
6
7 public:
8     int next(int price) {
9         int span = 1;
10        while (!st.empty() && st.top().first <= price) {
11            span += st.top().second;
12            st.pop();
13        }
14        st.push({price, span});
15        return span;
16    }
17 };
18
19 int main() {
20     StockSpanner spanner;
21     for (int p : {100, 80, 60, 70, 60, 75, 85}) {
22         cout << spanner.next(p) << " ";
23     }
24     cout << endl;
25     return 0;
26 }

```

Complexity Analysis

Time Complexity. Each day is pushed once and merged away (popped) at most once across its entire lifetime, giving

$O(1)$ amortized per day, $O(n)$ total for n days.

Space Complexity. $O(n)$ for the stack in the worst case (strictly decreasing prices, where every day stays on the stack).

Key Takeaway

When a monotonic-stack problem needs a *count* of absorbed elements rather than just the nearest boundary, store that count directly inside each stack entry and merge counts on pop – this avoids re-deriving the count from scratch every time and keeps the whole algorithm at amortized $O(1)$ per arrival, even for a problem (like this one) where the answer is a span length, not a position.

1.27 The Celebrity Problem

Problem Statement. There are n people at a party. A **celebrity** is a person whom **everyone else knows**, but who **knows no one else**. We have access to a function $knows(a, b)$ that returns whether person a knows person b . Find the celebrity if one exists (there is at most one), using as few calls to $knows$ as possible – ideally $O(n)$, not the $O(n^2)$ a full pairwise check would need.

Pattern Recognition

“Repeatedly eliminate one of two candidates based on a pairwise comparison, narrowing down to a single survivor” is a stack-driven elimination tournament: push every person as a candidate, then repeatedly pop two candidates and use a single $knows$ call to eliminate (rather than confirm) one of them, since the celebrity’s defining properties make it possible to always rule out at least one of any two candidates with a single check.

Understanding the problem carefully

Take any two candidates a and b , and ask $knows(a, b)$.

- If a knows b : then a cannot be the celebrity (a celebrity knows *no one*), so a is eliminated.
- If a does not know b : then b cannot be the celebrity (the celebrity must be known by *everyone*, but a does not know b), so b is eliminated.

Either way, a single call eliminates exactly one of the two candidates – never both, and never neither – so repeatedly applying this to pairs of remaining candidates narrows the field down to a single survivor using only $n - 1$ calls total.

Stack-Driven Elimination, Then One Verification Pass

Push every person $0, \dots, n - 1$ onto a stack. While more than one candidate remains: pop two, a and b . If $knows(a, b)$: push b back (eliminate a). Else: push a back (eliminate b). After this process, exactly one candidate remains – but it is only a *candidate*, not yet confirmed (it is possible that *no* celebrity exists at all). Verify the survivor by checking that *everyone else* knows them and that they know *no one else* – an additional $O(n)$ scan.

Why elimination narrows correctly, but still needs a final check

The elimination process guarantees that *if* a celebrity exists, they are never eliminated: a true celebrity, compared against any other candidate x , always satisfies $knows(celebrity, x) = \text{false}$ (the celebrity knows no one) and $knows(x, celebrity) = \text{true}$ (everyone knows the celebrity)

– so whichever of the two checks the algorithm happens to make, the celebrity is never the one eliminated. However, if *no* celebrity exists, the elimination process still produces *some* survivor (it always narrows to exactly one candidate, regardless of whether that candidate has the celebrity property) – which is why an explicit final verification pass is required to confirm or reject the survivor.

Algorithm

1. Push all n people onto a stack.
2. While the stack has more than one element:
 - (a) Pop a , then pop b .
 - (b) If $knows(a, b)$: push b . Else: push a .
3. Let *candidate* be the single remaining person.
4. Verify: for every other person x , check $knows(x, candidate) = \text{true}$ and $knows(candidate, x) = \text{false}$. If all checks pass, return *candidate*; otherwise, return “no celebrity exists.”

Dry run

Dry Run. Take $n = 4$ with *knows* such that person 2 is the true celebrity: everyone knows 2, and 2 knows no one.

Stack (top to bottom, after pushing 0, 1, 2, 3): top is 3.

- Pop $a = 3$, $b = 2$. Check $knows(3, 2)$: true (everyone knows 2). Push $b = 2$ (eliminate 3). Stack = [0, 1, 2].
- Pop $a = 2$, $b = 1$. Check $knows(2, 1)$: false (2 knows no one). Push $a = 2$ (eliminate 1). Stack = [0, 2].
- Pop $a = 2$, $b = 0$. Check $knows(2, 0)$: false. Push $a = 2$ (eliminate 0). Stack = [2].

Candidate = 2. Verification: every other person knows 2 (true), and 2 knows none of them (true). Confirmed: person 2 is the celebrity.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int findCelebrity(int n, function<bool(int, int)> knows) {
5     stack<int> st;
6     for (int i = 0; i < n; i++) st.push(i);
7
8     while (st.size() > 1) {
9         int a = st.top(); st.pop();
10        int b = st.top(); st.pop();
11
12        if (knows(a, b)) st.push(b); // eliminate a
13        else st.push(a);             // eliminate b
14    }
15
16    int candidate = st.top();
17    for (int x = 0; x < n; x++) {

```

```

18     if (x == candidate) continue;
19     if (!knows(x, candidate) || knows(candidate, x)) {
20         return -1; // no celebrity
21     }
22 }
23 return candidate;
24 }
25
26 int main() {
27     // Person 2 is the celebrity: everyone knows 2, 2 knows no one.
28     vector<vector<bool>> M = {
29         {0,1,1,1},
30         {0,0,1,0},
31         {0,0,0,0},
32         {0,1,1,0}
33     };
34     auto knows = [&](int a, int b) { return M[a][b]; };
35
36     cout << "Celebrity: " << findCelebrity(4, knows) << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. The elimination phase makes exactly $n - 1$ calls to *knows* (one per pop-pair, each removing one candidate from a starting pool of n), and the verification phase makes up to $2(n - 1)$ more calls, giving an overall time complexity of

$$O(n).$$

Space Complexity. $O(n)$ for the stack.

Key Takeaway

When a problem guarantees that comparing any two candidates always lets you safely eliminate *at least one* of them (never both, never neither), a stack-driven elimination tournament narrows n candidates to 1 in $O(n)$ comparisons. This is a different flavor of stack usage from the monotonic-stack family – here the stack just manages an elimination order, not a sorted skyline – and it is always worth pairing with an explicit final verification step whenever the narrowing process does not, by itself, guarantee the survivor actually has the desired property.

1.28 Sliding Window Maximum

Problem Statement. Given an array *nums* and a window size k , return an array containing the maximum element of every contiguous window of size k as it slides from the start of the array to the end.

Pattern Recognition

“Maximum (or minimum) over every sliding window” is the **monotonic deque** keyword from Section 1.1: a single index can remain a useful candidate for the maximum across *many* consecutive windows, but it must also eventually be evicted once the window slides past it – requiring removal from **both** ends (the back, when a better candidate arrives; the front, when the oldest index falls out of the window), which is exactly what a double-ended

queue (deque) provides.

Understanding the problem carefully

Maintain a deque of *indices* (not values), kept in decreasing order of their corresponding values from front to back, and restricted to indices currently inside the window. The front of the deque is always the index of the current window's maximum, because it is both the largest remaining value (by the decreasing-order invariant) and guaranteed to still be within the window (indices that fall out are actively removed from the front).

Monotonic Deque: Evict from the Back on Value, from the Front on Position

For each new index i : first, while the deque is non-empty and the value at its back index is $\leq \text{nums}[i]$, pop from the back (those indices can never again be the maximum of any window containing i , since i is both newer and at least as large – the same domination argument as next greater element). Push i to the back. Then, while the deque's front index is $\leq i - k$ (i.e. has fallen outside the current window), pop from the front. Once $i \geq k - 1$, the deque's front index is the maximum of the current window.

Why eviction from both ends preserves correctness

Eviction from the **back** is exactly the monotonic-stack domination argument from Section 1.15: a smaller, older index can never be the maximum once a newer, at-least-as-large value has arrived, anywhere within the window. Eviction from the **front** is new to this section and is required because, unlike a plain monotonic stack, indices here have a finite lifetime determined by the window's position – even an index that has never been dominated by value must still be discarded once it ages out of the window. Both evictions together guarantee that the deque, at every step, contains exactly the indices that are both currently in-window *and* not dominated by any later in-window value, kept in decreasing order – so its front is always correct.

Algorithm

1. Initialize an empty deque of indices and an empty result list.
2. For i from 0 to $n - 1$:
 - (a) While the deque is non-empty and $\text{nums}[\text{back index}] \leq \text{nums}[i]$: pop from the back.
 - (b) Push i to the back.
 - (c) While the deque's front index $\leq i - k$: pop from the front.
 - (d) If $i \geq k - 1$: append $\text{nums}[\text{front index}]$ to the result.
3. Return the result.

Dry run

Dry Run. Take $\text{nums} = [1, 3, -1, -3, 5, 3, 6, 7]$, $k = 3$.

- $i = 0$ (1): deque empty, push. Deque = $[0]$. $i < k - 1 = 2$, no output.
- $i = 1$ (3): back is index 0 (value $1 \leq 3$), pop. Push 1. Deque = $[1]$. $i < 2$, no output.
- $i = 2$ (-1): back is index 1 (value $3 \not\leq -1$), no pop. Push 2. Deque = $[1, 2]$. Front index $1 > i - k = -1$, no pop. $i = 2 \geq 2$: output $\text{nums}[1] = 3$.

- $i = 3$ (-3): no pop (back value $-1 \not\leq -3$). Push 3. Deque = $[1, 2, 3]$. Front index $1 \leq i - k = 0$? No ($1 > 0$). Output $nums[1] = 3$.
- $i = 4$ (5): back values $-3, -1, 3$ all ≤ 5 : pop all. Deque = $[],$ push 4. Deque = $[4]$. Front $4 \leq i - k = 1$? No. Output $nums[4] = 5$.
- $i = 5$ (3): back value $5 \not\leq 3$, no pop. Push 5. Deque = $[4, 5]$. Front $4 \leq i - k = 2$? No. Output $nums[4] = 5$.
- $i = 6$ (6): back values $3, 5$ both ≤ 6 : pop both. Push 6. Deque = $[6]$. Front $6 \leq i - k = 3$? No. Output $nums[6] = 6$.
- $i = 7$ (7): back value $6 \leq 7$, pop. Push 7. Deque = $[7]$. Front $7 \leq i - k = 4$? No. Output $nums[7] = 7$.

Result: $[3, 3, 5, 5, 6, 7]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> maxSlidingWindow(vector<int>& nums, int k) {
5     deque<int> dq; // stores indices
6     vector<int> result;
7
8     for (int i = 0; i < (int)nums.size(); i++) {
9         while (!dq.empty() && nums[dq.back()] <= nums[i]) {
10             dq.pop_back();
11         }
12         dq.push_back(i);
13
14         while (dq.front() <= i - k) {
15             dq.pop_front();
16         }
17
18         if (i >= k - 1) {
19             result.push_back(nums[dq.front()]);
20         }
21     }
22     return result;
23 }
24
25 int main() {
26     vector<int> nums = {1, 3, -1, -3, 5, 3, 6, 7};
27     for (int x : maxSlidingWindow(nums, 3)) cout << x << " ";
28     cout << endl;
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. Each index is pushed to the back once and popped (from either end) at most once across the whole scan, giving

$$O(n).$$

Space Complexity. $O(k)$ for the deque in the worst case.

Key Takeaway

A monotonic deque is a monotonic stack with one extra capability: aging out elements from the opposite end once they fall outside a window. Any time a problem combines “sliding window” with “maximum/minimum within the window,” this is the data structure to reach for, rather than re-scanning each window from scratch (which would cost $O(nk)$) or using a heap (which would cost $O(n \log k)$ but require extra bookkeeping to evict stale entries lazily).

1.29 LRU Cache

Problem Statement. Design a cache with a fixed capacity supporting $get(key)$ (return the value if present, else -1 , and mark the key as recently used) and $put(key, value)$ (insert or update a key, marking it recently used; if the cache is full and a new key must be inserted, evict the **least recently used** key first). Both operations must run in $O(1)$.

Pattern Recognition

“*Most/least recently used, with fast lookup*” is the keyword combination from Section 1.1: $O(1)$ lookup by key demands a hash map, while $O(1)$ “move this to the most-recent end” and “evict the least-recent end” demands a structure with fast removal and insertion at *both* ends and from the *middle* – which a plain array or singly linked list cannot offer, but a **doubly linked list** can.

Understanding the problem carefully

A hash map alone gives $O(1)$ lookup but no notion of “recency order.” A queue alone gives recency order but $O(n)$ lookup by key, and moving an arbitrary key to the front of recency would require knowing its position. Combining both is the standard fix: a **doubly linked list** maintains recency order (most recently used at one end, least recently used at the other), and a **hash map from key to that key’s node** gives $O(1)$ lookup – crucially, because the list is *doubly* linked, removing a node found via the hash map (to move it or evict it) takes $O(1)$, since we have direct pointers to its neighbors without needing to traverse from either end.

Doubly Linked List + Hash Map

Maintain a doubly linked list with two sentinel nodes, *head* and *tail*, where the side nearest *head* represents *most* recently used and the side nearest *tail* represents *least* recently used. A hash map stores $key \rightarrow node\ pointer$. On $get(key)$: if present, unlink the node from its current position and re-insert it right after *head* (marking it most recent), then return its value; if absent, return -1 . On $put(key, value)$: if the key exists, update its value and move its node to just after *head* (same as a successful get). If the key is new and the cache is at capacity, remove the node just before *tail* (the least recently used) along with its hash map entry; then insert a new node for this key just after *head*.

Why doubly linked, not singly linked, is essential

The operation that makes this problem hard is not insertion or removal at the *ends* (a singly linked list with head and tail pointers could handle that, as in Section 1.4) – it is removal from an **arbitrary middle position**, which happens every time an already-cached key is accessed again and must be promoted to most-recently-used. Removing a node from the middle of a singly linked list requires knowing its *predecessor*, which means traversing from the head – an

$O(n)$ operation. A doubly linked list stores each node's predecessor pointer directly, so the hash map's stored node pointer alone is enough to unlink it in $O(1)$, with no traversal needed at all.

Algorithm

1. Maintain sentinel *head* and *tail* nodes, a hash map $key \rightarrow node$, and a capacity limit.
2. $GET(key)$: if absent, return -1 . Else: unlink the node; re-insert it right after *head*; return its value.
3. $PUT(key, value)$: if present, update the node's value and move it to right after *head* (as in GET). If absent and at capacity: remove the node just before *tail* (and its map entry). Create a new node, insert it right after *head*, and add it to the map.

Dry run

Dry Run. Capacity 2. $PUT(1,1)$, $PUT(2,2)$, $GET(1)$, $PUT(3,3)$ (evicts LRU), $GET(2)$.

- $Put(1,1)$: list (MRU $\rightarrow$ LRU) = [1].
- $Put(2,2)$: list = [2, 1].
- $Get(1)$: found; move to front. List = [1, 2]. Returns 1.
- $Put(3,3)$: cache full (2 keys); evict LRU end (2). List = [3, 1].
- $Get(2)$: key 2 was evicted; returns -1 .

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class LRUCache {
5      struct Node {
6          int key, value;
7          Node* prev = nullptr;
8          Node* next = nullptr;
9          Node(int k, int v) : key(k), value(v) {}
10     };
11
12     int capacity;
13     Node* head; // sentinel: head->next is most recently used
14     Node* tail; // sentinel: tail->prev is least recently used
15     unordered_map<int, Node*> map;
16
17     void remove(Node* node) {
18         node->prev->next = node->next;
19         node->next->prev = node->prev;
20     }
21
22     void insertAfterHead(Node* node) {
23         node->next = head->next;
24         node->prev = head;
25         head->next->prev = node;
26         head->next = node;
27     }

```

```

28
29 public:
30     LRUCache(int cap) : capacity(cap) {
31         head = new Node(-1, -1);
32         tail = new Node(-1, -1);
33         head->next = tail;
34         tail->prev = head;
35     }
36
37     int get(int key) {
38         if (!map.count(key)) return -1;
39         Node* node = map[key];
40         remove(node);
41         insertAfterHead(node);
42         return node->value;
43     }
44
45     void put(int key, int value) {
46         if (map.count(key)) {
47             Node* node = map[key];
48             node->value = value;
49             remove(node);
50             insertAfterHead(node);
51             return;
52         }
53         if ((int)map.size() == capacity) {
54             Node* lru = tail->prev;
55             remove(lru);
56             map.erase(lru->key);
57             delete lru;
58         }
59         Node* node = new Node(key, value);
60         insertAfterHead(node);
61         map[key] = node;
62     }
63 };
64
65 int main() {
66     LRUCache cache(2);
67     cache.put(1, 1);
68     cache.put(2, 2);
69     cout << cache.get(1) << endl; // 1
70     cache.put(3, 3); // evicts key 2
71     cout << cache.get(2) << endl; // -1
72     return 0;
73 }

```

Complexity Analysis

Time Complexity. $O(1)$ for both *get* and *put* – hash map lookup, plus a constant number of pointer updates for the doubly linked list operations. **Space Complexity.** $O(\text{capacity})$ for the nodes and hash map.

Key Takeaway

“Fast lookup by key” plus “fast reordering by recency, including removal from the middle” is the signature combination that demands a hash map paired with a *doubly* linked list, not

a singly linked one – the doubly linked structure is what converts an $O(n)$ traversal-to-find-predecessor into an $O(1)$ direct unlink. The same pairing, with a different eviction rule, solves LFU Cache next.

1.30 LFU Cache

Problem Statement. Design a cache with a fixed capacity supporting $get(key)$ and $put(key, value)$, both in $O(1)$, where eviction (when the cache is full and a new key must be inserted) removes the **least frequently used** key; if multiple keys share the lowest frequency, the **least recently used** among them is evicted.

Pattern Recognition

LFU is LRU (Section 1.29) with a second tie-breaking dimension added: eviction priority is now *(frequency, recency)*, not recency alone. The keyword to notice: whenever a tie-breaking rule is layered on top of a structure you already have working (here, LRU’s hash-map-plus-doubly-linked-list), try **bucketing** by the primary key (frequency) and reusing the existing structure (an LRU list) as the secondary tie-breaker *within* each bucket, rather than designing an entirely new structure from scratch.

Understanding the problem carefully

Group keys by their current access frequency into separate buckets. Each bucket, internally, only ever needs to break ties by recency among keys that share that exact frequency – which is exactly the LRU Cache substructure from Section 1.29, reused once per distinct frequency value. Eviction always targets the *lowest* currently-occupied frequency bucket, and within it, removes that bucket’s least-recently-used key (the LRU substructure’s own eviction rule, applied locally).

A Hash Map of Frequency Buckets, Each an LRU List

Maintain: a hash map *keyData* from key to $(value, freq)$; a hash map *freqBuckets* from frequency to a doubly linked list (an LRU list) of keys currently at that frequency, ordered by recency within the bucket; and a single integer *minFreq*, tracking the smallest frequency currently occupied by any key. On every access to an existing key (via *get* or an updating *put*): remove it from its current frequency bucket (inserting it at the front, i.e. most-recent position, of the *next* frequency bucket up), incrementing its stored frequency; if its old bucket becomes empty and was the minimum, increment *minFreq*. On inserting a brand-new key when the cache is full: evict the least-recently-used key from the *minFreq* bucket, then insert the new key into the frequency-1 bucket and reset $minFreq \leftarrow 1$.

Why tracking *minFreq* explicitly avoids an expensive search

Without an explicit running *minFreq*, finding “the lowest currently-occupied frequency” would require scanning all buckets, costing more than $O(1)$. Because frequency only ever **increases** by exactly 1 per access, and a brand-new key always enters at frequency 1, we can maintain *minFreq* incrementally with simple, local updates: it can only decrease (to 1) when a new key is inserted, and it can only increase when the bucket it currently points to becomes completely empty (because the key that just left was the last one at that frequency, and every other key’s frequency is, by definition, at least *minFreq*, so the new minimum must be $minFreq+1$ in that

specific case). This local bookkeeping replaces a potentially expensive global search with $O(1)$ updates.

Algorithm

1. **TOUCH**(*key*) (shared helper for get and updating put): look up (*value*, *f*) in *keyData*; remove *key* from *freqBuckets*[*f*]; if that bucket is now empty and $f = \text{minFreq}$, increment *minFreq*; insert *key* at the front of *freqBuckets*[$f + 1$]; update *keyData*[*key*] $\leftarrow$ (*value*, $f + 1$).
2. **GET**(*key*): if absent, return -1 . Else: **TOUCH**(*key*); return its value.
3. **PUT**(*key*, *value*): if present, update the stored value and call **TOUCH**(*key*). Otherwise, if at capacity: evict the least-recently-used key from *freqBuckets*[*minFreq*] (and its *keyData* entry). Insert *key* at the front of *freqBuckets*[1]; set *keyData*[*key*] $\leftarrow$ (*value*, 1); set *minFreq* $\leftarrow$ 1.

Dry run

Dry Run. Capacity 2. **PUT**(1,1), **PUT**(2,2), **GET**(1) (freq of 1 becomes 2), **PUT**(3,3) (cache full; evict lowest-frequency key, which is 2, frequency 1), **GET**(2).

- **Put**(1,1): bucket[1] = [1]. minFreq = 1.
- **Put**(2,2): bucket[1] = [2, 1] (2 most recent). minFreq = 1.
- **Get**(1): touch 1 (freq 1 $\rightarrow$ 2). Remove from bucket[1]: bucket[1] = [2] (not empty, minFreq stays 1). Insert into bucket[2] = [1].
- **Put**(3,3): cache full (2 keys). Evict from bucket[*minFreq*=1]'s LRU end: bucket[1] = [2], evict 2. bucket[1] now empty. Insert 3 into bucket[1] = [3]. minFreq reset to 1.
- **Get**(2): key 2 was evicted; returns -1 .

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class LFUCache {
5      int capacity, minFreq;
6      unordered_map<int, pair<int,int>> keyData; // key -> (value, freq)
7      unordered_map<int, list<int>> freqBuckets; // freq -> keys (front
           = most recent)
8      unordered_map<int, list<int>::iterator> keyIter; // key -> its
           position in its bucket
9
10     void touch(int key) {
11         auto [value, freq] = keyData[key];
12         freqBuckets[freq].erase(keyIter[key]);
13         if (freqBuckets[freq].empty()) {
14             freqBuckets.erase(freq);
15             if (freq == minFreq) minFreq++;
16         }
17         freqBuckets[freq + 1].push_front(key);
18         keyIter[key] = freqBuckets[freq + 1].begin();
19         keyData[key] = {value, freq + 1};
20     }

```

```

21
22 public:
23     LFUCache(int cap) : capacity(cap), minFreq(0) {}
24
25     int get(int key) {
26         if (!keyData.count(key)) return -1;
27         touch(key);
28         return keyData[key].first;
29     }
30
31     void put(int key, int value) {
32         if (capacity == 0) return;
33
34         if (keyData.count(key)) {
35             keyData[key].first = value;
36             touch(key);
37             return;
38         }
39
40         if ((int)keyData.size() == capacity) {
41             int evictKey = freqBuckets[minFreq].back();
42             freqBuckets[minFreq].pop_back();
43             keyData.erase(evictKey);
44             keyIter.erase(evictKey);
45         }
46
47         keyData[key] = {value, 1};
48         freqBuckets[1].push_front(key);
49         keyIter[key] = freqBuckets[1].begin();
50         minFreq = 1;
51     }
52 };
53
54 int main() {
55     LFUCache cache(2);
56     cache.put(1, 1);
57     cache.put(2, 2);
58     cout << cache.get(1) << endl; // 1
59     cache.put(3, 3); // evicts key 2
60     cout << cache.get(2) << endl; // -1
61     return 0;
62 }

```

Complexity Analysis

Time Complexity. $O(1)$ for both *get* and *put* – every step is a hash map lookup or a constant number of doubly linked list operations, with *minFreq* maintained incrementally rather than recomputed. **Space Complexity.** $O(\text{capacity})$ across the hash maps and linked lists.

Key Takeaway

This closing problem of the chapter is a deliberate capstone, much like Median of a Row-wise Sorted Matrix was for the Binary Search chapter: it requires recognizing that a new, seemingly more complex requirement (frequency-based eviction) can be built by *bucketing* on the new dimension and reusing an already-solved structure (LRU's doubly linked list) as the tie-breaker within each bucket – composition of known pieces, rather than a wholly

new design, solves the harder problem.